

●Week 1

1. 課程介紹

(1) What is Computer Graphics

Computer graphics is a field of study and practice that involves creating, manipulating, and displaying visual content using computers.

Including 2D and 3D graphics, animation, digital image processing, and virtual reality

(2) Applications of CG

Creating animated movies and videos games (designing user interfaces)

→總結：電腦圖學是一門重要的研究學科，探討學習圖形、動畫和遊戲的基本原理

2. 教學目標：

To learn fundamental of 2D and 3D computer graphics.

To learn how to create 2D, 3D interactive graphics applications in HTML5 web browsers.

3. 說明課程評分：

(1) 工作日誌：學習心得與程式寫作紀錄(練習清單、程式、成果、心得)(60-80%)

(2) 作業(20-40%)

★心得：

了解該堂課該學習的目標、評分與初步了解電腦圖學的定義和應用。

●Week 2

※2D、3D 介紹

1. 2D Graphics

(1) Canvas (using javascript)

(2) SVG (Scalable Vector Graphics)

2. 3D Graphics

(1) WebGL

(2) CSS 3D

※HTML 5

1. HTML (tag 標籤用於主題、來源)

2. CSS (style 定義網頁的各式外觀背景顏色)

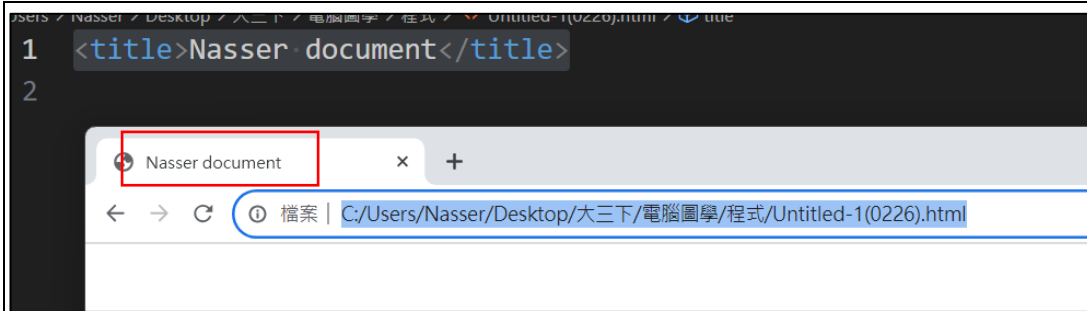
3. JavaScript (behavior 用程式語言去畫圖)

→不同軟體應用不同的程式語言來畫圖

※HTML 程式操作

1.

```
<title>Nasser document</title>
```

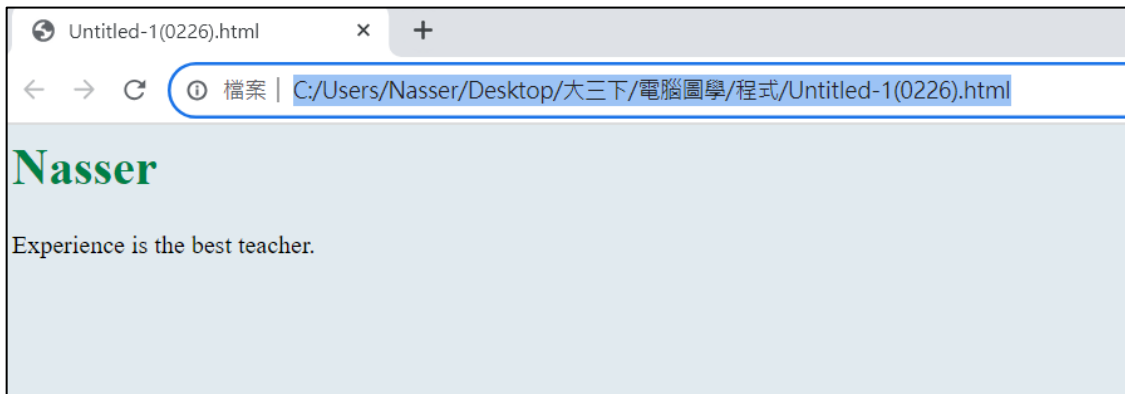


2.HTML 語法操作

```
<head>
<style>
body {
  background-color: rgb(225, 234, 239);
}

h1 {
  color: rgb(0, 128, 70);
}
</style>
</head>
<body>
<h1>Nasser</h1>
<p>Experience is the best teacher.</p>
</body>
```

背景顏色



3.

```
<!DOCTYPE html>
<html>
<head>
<script>
function doCircle(evt)
```

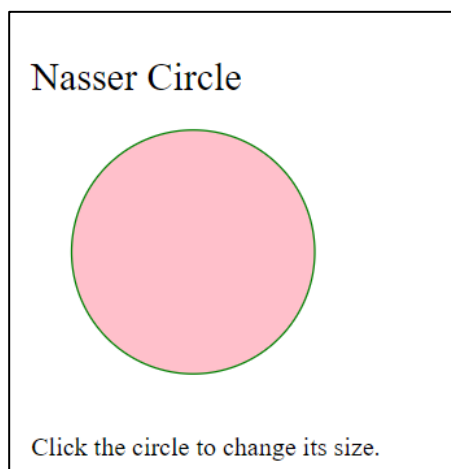
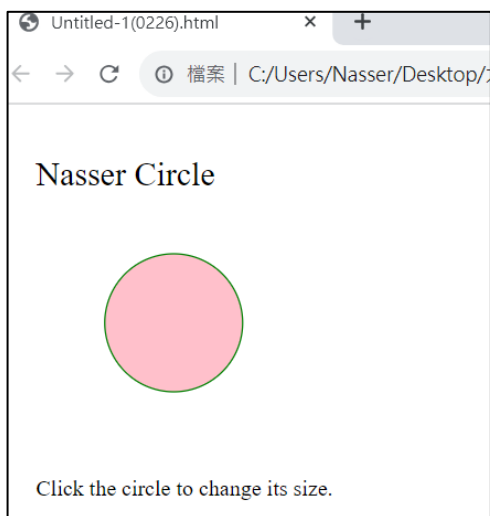
```

{
var theCircle = evt.target;
var radius = theCircle.getAttribute("r");
if (radius == 50)
radius = 75; else radius = 50;
theCircle.setAttribute("r",radius);
}
</script>
</head>
<body>
<svg width="300px" height="300px">
<text x="25" y="50" font-size="24">Nasser Circle</text>
<text x="25" y="275">Click the circle to change its size.</text>
<circle cx="125" cy="150" r="50" fill="pink" stroke="green" stroke-width="5"
onclick="doCircle(evt)" />
</svg>
</body>
</html>

```

點擊圓，會改變圓的大小

文字位置、圓的大小及顏色



4. Draw Text: font, fill Style

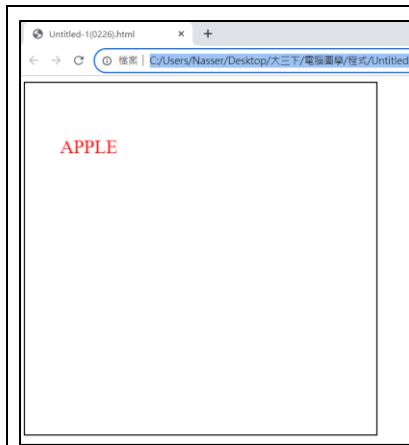
```

<canvas id="myCanvas" width="500" height="500" style="border:2px solid"></canvas>
<script>
var canvas = document.getElementById("myCanvas");
var context = canvas.getContext("2d");
context.font = "20pt Ariel"; context.fillStyle = "red"; context.fillText("APPLE",50, 100);
</script>

```

外框大小及顏色

文字的大小、顏色、位置



5. Draw a Circle

```

<html>
<body>
<canvas id="myCanvas" width="500" height=" " style="border:2px solid #dd1a1a";></canvas>

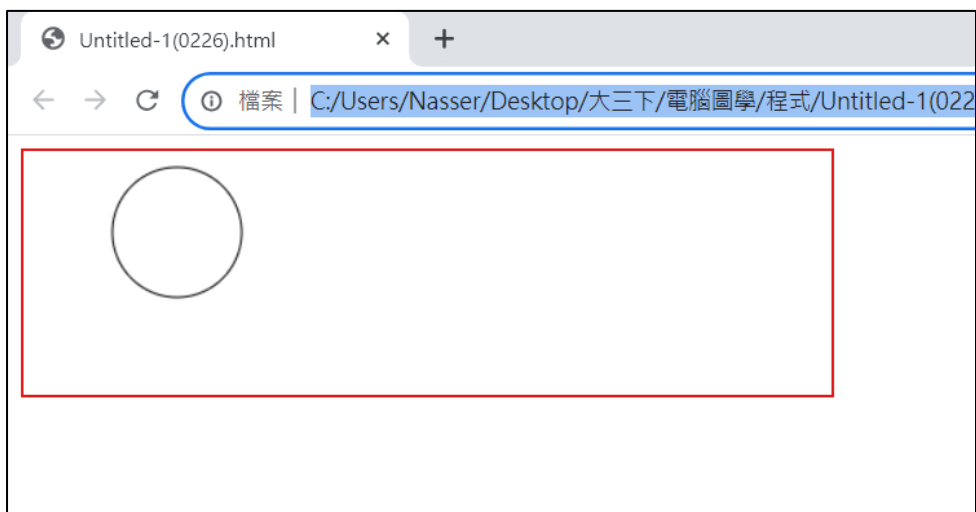
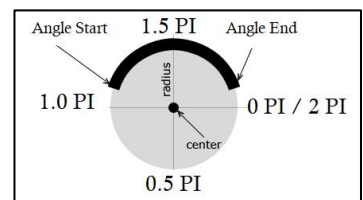
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.beginPath();
ctx.arc(95,50,40,0,2*Math.PI);
ctx.stroke();
</script>

</body>
  
```

外框線條大小及顏色

圓的大小、位置

`arc(x, y, r, startangle, endangle)`



6. Draw a rectangle

```

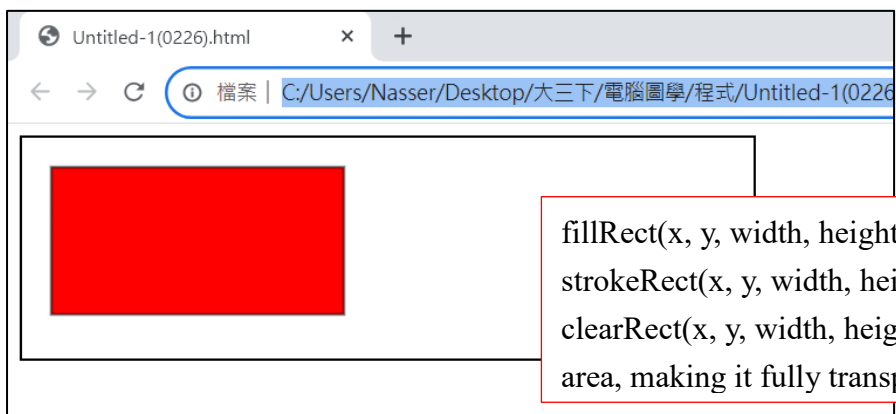
<html>
<body>
<canvas id="myCanvas" width="500" height=" " style="border:2px solid #000000" ;></canvas>

<script>
const canvas = document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");
ctx.fillStyle="#FF0000";
ctx.fillRect(20, 20, 200, 100);
ctx.strokeRect(20,20,200,100);
</script>
</body>

```

外框大小及顏色

矩形大小、填滿、顏色、外框



fillRect(x, y, width, height) draws a filled rectangle
 strokeRect(x, y, width, height) draws a rectangular outline
 clearRect(x, y, width, height) clears the specified rectangle area, making it fully transparent

★心得：

初步了解 2D、3D 應用，而本次課程最重要的部分是 HTML5 的介紹，講解 HTML5 的基本語法，開始應用於圖形上，除了基本的文字、外框、顏色等語法撰寫，還有矩形及圓型的程式撰寫，能了解程式代表的意涵與能夠呈現之圖案。

●Week 3

1. 複習 html 的語法，使用 html 裡的 canvas 來建立畫布

2. 程式操作

(1) Draw a Circle

<html>

<body>

<canvas id="myCanvas" width="500" height="300"></canvas>

<script>

var c = document.getElementById("myCanvas");

var ctx = c.getContext("2d");

建立畫布

ctx.beginPath();

ctx.arc(95,50,40,0,1.5*Math.PI);

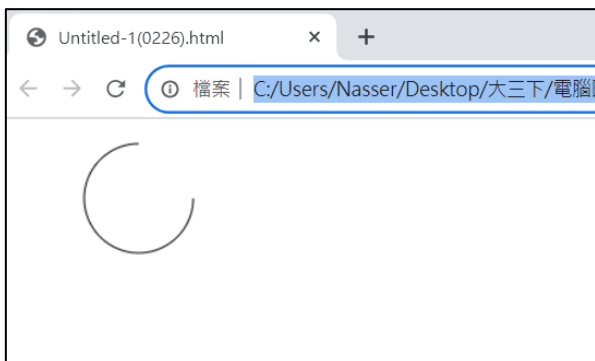
arc(x, y ,r, startangle, endangle)

ctx.stroke();

</script>

</body>

</html>



(2) Grandients 漸層

<html>

<body>

<canvas id="myCanvas" width="500" height="300"></canvas>

<script>

var c = document.getElementById("myCanvas");

var ctx = c.getContext("2d");

// Create gradient

var grd=ctx.createLinearGradient(0,0,150,0);

grd.addColorStop(0,"red");

```

grd.addColorStop(1,"blue");
grd.addColorStop(1,"green");
grd.addColorStop(1,"white");

```

可以增加顏色，不同效果

```

// Fill with gradient
ctx.fillStyle=grd;
ctx.fillRect(10,10,150,80);

```

```

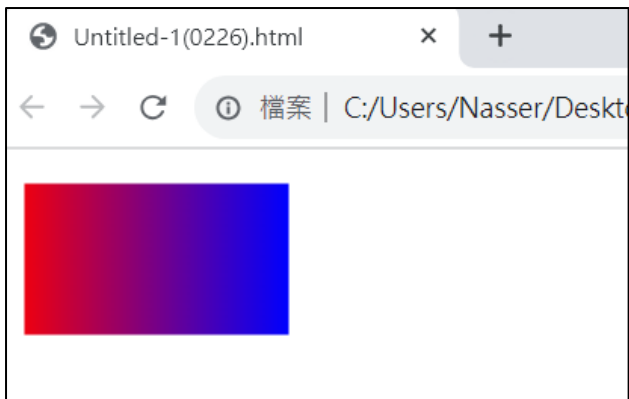
</script>
</body>

```

```

</html>

```



(3) Radial Gradient

```

<html>

```

```

<body>

```

```

<canvas id="myCanvas" width="500" height="300"></canvas>

```

```

<script>

```

```

var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");

```

```

var g = ctx.createRadialGradient(150, 150, 50, 150, 150, 100);
g.addColorStop(0, "red");
g.addColorStop(0, "green");
g.addColorStop(0.5, "yellow");

```

圓的大小及圓弧顏色

```

g.addColorStop(1.0, "blue");
ctx.fillStyle = g;
ctx.fillRect(20, 20, 260, 260);

```

外框方形的大小及顏色

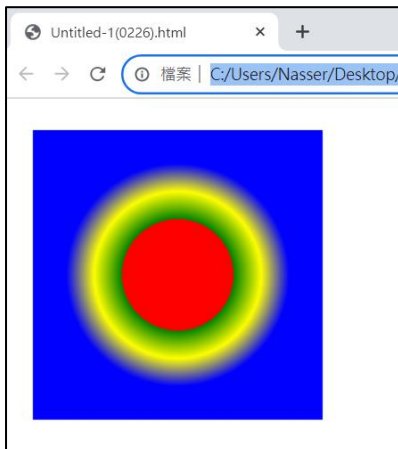
```

</script>

```

</body>

</html>



(4) Load image

<html>

放入的圖片/圖片的網址

<body>

<canvas id="myCanvas" width="240" height="297" style="border:1px solid grey;"></canvas>

<script>

var c = document.getElementById("myCanvas");

var ctx = c.getContext("2d");

const canvas = document.getElementById("myCanvas");

const ctx = canvas.getContext("2d");

const img = document.getElementById("logo");

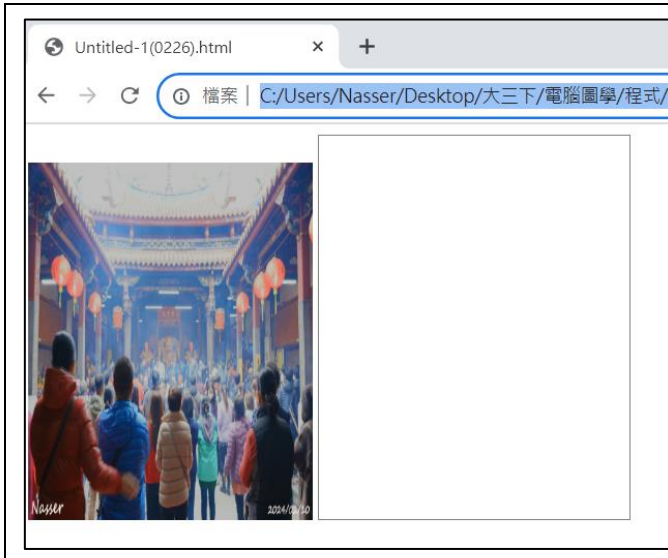
ctx.drawImage(img, 10, 10);

圖片的大小

</script>

</body>

</html>



(5) Bouncing Ball 彈跳球 (L2)

<html>

<body>

```
<canvas id="myCanvas" width="500" height="500" style="border:1px solid
#000000;"></canvas>
```

<script>

class Ball {

constructor(x, y, r, color) {

this.x = x;

this.y = y; this.r = r;

this.color = color;

this.dx = random(-3, 3) + 0.001; this.dy = random(-3, 3) + 0.001;

}

draw(g) {

g.beginPath();

g.arc(this.x, this.y, this.r, 0, Math.PI * 2); g.fillStyle = this.color;

g.fill();

}

move() {

this.x = this.x + this.dx;

if (this.x > width || this.x < 0) {

this.dx = -this.dx; this.x = this.x + this.dx;

}

this.y = this.y + this.dy;

設定球的 x,y 位置，半徑(r)，與顏色 (color)，與球的水平(dx)垂直(dy)移動量

球的移動量

球的移動球碰到牆壁，反彈 (反方向跑)

球的繪製，由球的 x,y 位置，半徑，與顏色決定球的外觀

計算球的移動，當碰到畫布 (canvas)邊緣時，球會反彈，進行反向運動 (dx -> -dx, dy -> -dy)

```

        if (this.y > height || this.y < 0) {
            this.dy = -this.dy; this.y = this.y + this.dy;
        }
    }
}

var canvas = document.getElementById("myCanvas");
var ctx = canvas.getContext("2d");

var width = canvas.width;
var height = canvas.height;

function random(min, max) {
    return Math.floor(Math.random() * (max - min)) + min;
}

var balls = [];
for (i = 0; i < 50; i++) {
    x = random(0, width); y = random(0, height);
    r = 30 + random(0, 10);
    color = 'rgba(' + random(0, 255) + "," + random(0, 255) + "," + random(0, 255) + "," +
Math.random() + ")";
    balls.push(new Ball(x, y, r, color));
}
render();
function render() {
    ctx.clearRect(0, 0, width, height);

    for (i = 0; i < balls.length; i++) {
        balls[i].draw(ctx);
        balls[i].move();
    }
    window.requestAnimationFrame(render);
}
</script>
</body>
</html>

```

隨機數

產生隨機 RGBA 顏色，範圍從暗到亮(0~255)
r=30 是球的半徑

分別是 R G B A

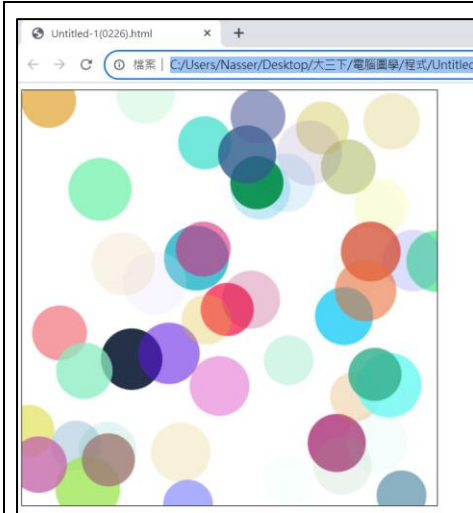
將所有球(balls)畫在畫布裡，
是一個 recursive function，會不斷的叫自己而達成動畫影格的繪製

清除上一影格的畫面，避免畫面殘留的問題

將每一個球畫出，並進行下一步的移動

呼叫自己

約 1/60 秒後會呼叫 render，進行下一個畫面的繪製，而形成動畫



(6) EventListener

```
<html>
```

```
<body>
```

```
  <canvas id="myCanvas" width="500" height="500" style="border:1px solid  
#000000;"></canvas>
```

```
  <button id="myBtn">Try it</button>
```

```
  <script>
```

```
    document.getElementById("myBtn").addEventListener("click", myFunction);
```

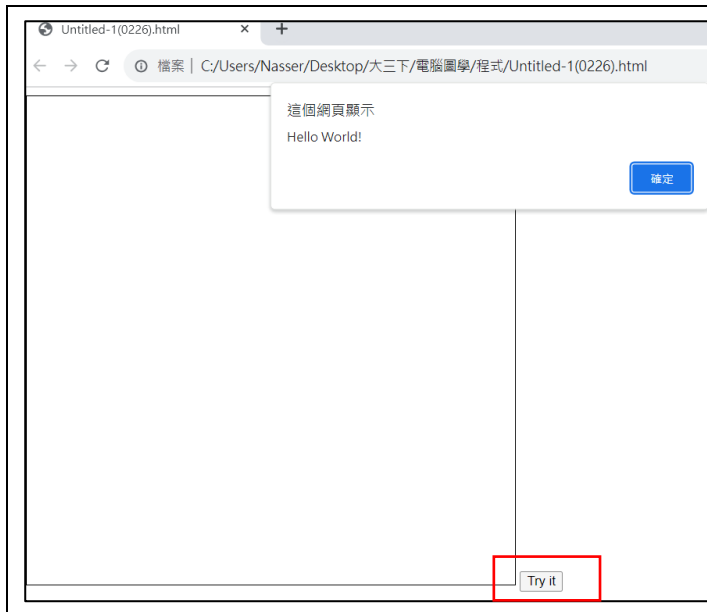
```
    function myFunction() {  
      alert("Hello World!");  
    }
```

```
  </script>
```

```
</body>
```

```
</html>
```

addEventListener 可以監控事件(click, mouseout…), 若發生可以執行 myFunction()



mouse over 滑鼠游標滑入 可出現
mouse out
滑鼠游標滑入再滑出 可出現
click 點擊

★心得：

上次課程結束後，還有些程式部分不理解，但在這次上課，老師有重新複習一次所有的觀念，把不懂的部分解決。此次上課除了複習上次所教的觀念外，還特別提到漸層塗色、如何放入圖片及彈跳球如何產生的程式，特別是彈跳球的程式較為複雜，除了融合上次所學，也包含顏色、碰撞和該如何消除殘留影格的問題，是一個完整的練習過程。對於這次上課，內容皆為簡易，複習上課所教的觀念，能理解概念並實作出來。

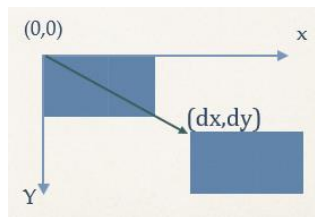
Week4

1. 2D Transformation 講解

- (1) Linear Transformation→常應用在貼圖上
- (2) Standard transformation used in computer graphics
 - a. translation (移動量_點到點的移動)→non-linear
 - b. rotation
 - c. scaling
 - d. shearing
- (3) Represent coordinate in 2D dimension with 3-vector
 $(x, y) \Rightarrow (x, y, w)$ 從非線性變成線性(translation)

2. Translation(dx,dy)

$$\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}$$



3. 操作演練

(1) Rotation

```
<canvas id="myCanvas" width="500" height="500"></canvas>
```

```
<script>
```

```
    canvas = document.getElementById("myCanvas");
```

```
    g = canvas.getContext("2d");
```

```
    g.translate(100, 100);
```

```
    g.fillStyle = 'blue';
```

```
    g.fillRect(0, 0, 100, 80);
```

```
    g.rotate(Math.PI / 3);
```

```
    g.fillStyle = 'red';
```

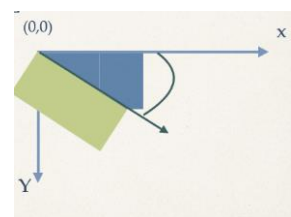
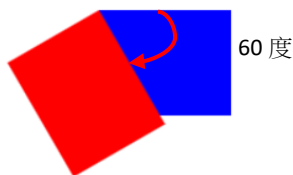
```
    g.fillRect(0, 0, 100, 80);
```

```
</script>
```

藍色方塊

移動量(旋轉指令)

紅色方塊



(2) Reflection 鏡射

```

<canvas id="myCanvas" width="500" height="500"></canvas>
<script>
    canvas = document.getElementById("myCanvas");
    g = canvas.getContext("2d");

    g.translate(100, 100);
    g.fillStyle = 'red';
    g.fillRect(0, 0, 100, 80);
    g.scale(-1, 1);
    g.fillStyle = 'yellow';
    g.fillRect(0, 0, 100, 80);
</script>

```

鏡射方向(負)



(3) 移動及旋轉

```

<canvas id="mycanvas" width="300" height="300" style="border:1px solid
#000000;"></canvas>
<html>

<body>
    <script>
        var canvas = document.getElementById("mycanvas");
        var ctx = canvas.getContext("2d");

        function drawRect(context, color) {
            context.fillStyle = color;
            context.fillRect(0, 0, 100, 30);
        }

        function rotateDeg(context, deg) {
            var rad = deg * Math.PI / 180;
            context.rotate(rad);
        }

        drawRect(ctx, "red");
        ctx.save(); // 儲存狀態

        ctx.translate(100, 30); // 平行移動位置
    </script>

```

```

rotateDeg(ctx, 45); // 旋轉 45 度
drawRect(ctx, "blue");
ctx.restore(); // 復原狀態
// (3)指定移動與旋轉並描繪
ctx.save(); // 儲存狀態

ctx.translate(200, 50); // 平行移動位置

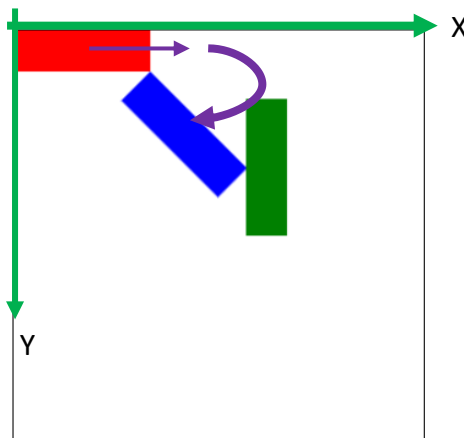
rotateDeg(ctx, 90); // 旋轉 90 度
drawRect(ctx, "green");
ctx.restore(); // 復原狀態

```

```
</script>
```

```
</body>
```

```
</html>
```



紅色方塊平行移動 $x=100, y=33$ 並旋轉 45 度，變成藍色，接著移動狀態復原至紅色位置(儲存)。再來平行移動 $x=200, y=50$ ，並旋轉 90 度，變成綠色，接著移動狀態復原至紅色位置(儲存)。

(4) firework

```
<canvas id="mycanvas" width="800" height="500" style="border:1px solid ;"></canvas>
```

```
<html>
```

```
<body>
```

```
<script>
```

```

var canvas = document.getElementById("mycanvas");
var ctx = canvas.getContext("2d");
class Particle {
  constructor(x, y, color) {
    this.x = x;
    this.y = y;
    this.color = color;

    this.vx = (Math.random() - 0.5) * 8; //粒子的初始速度

```

```

        this.vy = (Math.random() - 0.5) * 8; //粒子的初始速度

        this.alpha = 1;
        this.friction = 0.99;
    }
    draw() {
        ctx.globalAlpha = this.alpha;
        ctx.beginPath();
        ctx.arc(this.x, this.y, 2, 0, Math.PI * 2);
        ctx.fillStyle = this.color;
        ctx.fill();
    }
    update() {
        this.vx *= this.friction;
        this.vy *= this.friction;
        this.x += this.vx;
        this.y += this.vy;

        this.alpha -= 0.01; //alpha 不透明度
    }
} //以上代表(x, y), 顏色, 速度, 不透明度(alpha), 摩擦 friction.

```

```

class Firework {
    constructor(x, y, color) {
        this.x = x;
        this.y = y;
        this.color = color;
        this.vx = 0;
        this.vy = Math.random() * -2.5 - 0.5;
        this.particles = [];

        this.lifespan = 180; //高空爆炸的生命時間

        this.hasExploded = false;
    }
    draw() {
        ctx.beginPath();
        ctx.arc(this.x, this.y, 3, 0, Math.PI * 2);
        ctx.fillStyle = this.color;
        ctx.fill();
    }
}

```

```

    }

    explode() {
        for (var i = 0; i < 50; i++) {
            this.particles.push(new Particle(this.x, this.y, this.color));
        }
    } //迴圈(每次產生一個粒子，擴散的動作)

    update() {
        this.lifespan--;

        if (this.lifespan <= 0 && !this.hasExploded) {
            this.explode();

            this.vx = 0; this.vy = 0; //爆炸中心點不動

            this.hasExploded = true;
        } else if (this.lifespan > 0) {
            this.y += this.vy;
        } //高空爆炸(從 180~0)

        for (var i = 0; i < this.particles.length; i++) {
            this.particles[i].update();
            this.particles[i].draw();
        } //往外慢慢擴散
    }
}

fireworks = [];

function animate() {
    ctx.fillStyle = 'rgba(0, 0, 0, 0.1)';
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    for (var i = 0; i < fireworks.length; i++) {
        fireworks[i].update();
        fireworks[i].draw();
        if (fireworks[i].lifespan <= 0 && fireworks[i].particles[0].alpha <= 0) {
            fireworks.splice(i, 1);
        }
    }
}

```

```
};

if (Math.random() < 0.015) {

    const x = Math.random() * canvas.width; //煙火產生位置隨機

    const color = 'rgb(' + Math.random() * 255 + ',' + Math.random() * 255 + ',' +
Math.random() * 255 + ')'; //煙火顏色隨機

    fireworks.push(new Firework(x, canvas.height, color));
}

requestAnimationFrame(animate);

//利用 requestAnimationFrame 製造能放煙火的迴圈

}

animate();

</script>
</body>

</html>
```



第 4 個練習 (firework) 配合作業一 (Hw1)

※Hw1(作業)

Design interactive particle effects such as fireworks, particles using canvas drawing.

Particle can be any shape like circle, star, ..

Show you highlights

Please cite the source of reference

★心得：

這次上課除了複習上次課程所教的之外，也繼續教基本的移動、旋轉及鏡射的練習，而本次最

電腦圖學 Week 3-4

S1061103 許鉦偉

重要的為煙火的發射講解。從粒子的建立、顏色、速度和不透明度到如何建立煙火與動畫，利用迴圈、生命值等設定隨機值(ex:顏色、煙火發射的位置)。

上課較難理解的部分即是煙火的動畫，且有待改善，因為煙火動畫有時會卡頓，因此下次上課有待修改此程式，並且重新一一說明。

●Week 5

1. 複習上次所學的煙火，並修改卡頓的問題

2. HTML 演練及講解

1. Digital Clock

```
<html>
```

```
<body>
```

```
<canvas id="mycanvas" width="800" height="500" style="border:1px solid ;"></canvas>
```

```
<script>
```

```
var canvas;
```

```
var g;
```

```
var radius;
```

```
init();
```

```
function init() {
```

```
    canvas = document.getElementById("mycanvas");
```

```
    g = canvas.getContext('2d');
```

```
    radius = canvas.height / 2 * 0.9;
```

```
    setInterval(drawClock, 1000);           //設定 drawClock 呼叫間隔, 每 1sec 重畫時間
```

```
}
```

```
//drawClock();
```

```
function drawClock() {
```

```
    g.save();
```

```
    g.clearRect(0, 0, canvas.width, canvas.height);
```

```
    g.translate(canvas.width / 2, canvas.height / 2);
```

```
    drawFace();
```

```
    drawText();
```

```
    drawTime();
```

```
    g.restore();
```

```
}
```

畫 clock, 儲存狀態→移到 clock 中心→畫
出 clock(drawFace, drawText, drawTime)

```
drawFace();
```

```
function drawFace() {
```

```
    g.beginPath();           // outer circle(時鐘的外框繪製)
```

```
    g.arc(0, 0, radius, 0, 2 * Math.PI);
```

```
    g.fillStyle = 'rgb(30,30,120,0.5)';
```

```
    g.fill();
```

```

grad = g.createRadialGradient(0, 0, radius * 0.90, 0, 0, radius * 1.1);
grad.addColorStop(0, '#333');
grad.addColorStop(0.5, 'rgb(30,30,70,0.5)');
grad.addColorStop(1, '#333');
g.strokeStyle = grad;
g.lineWidth = radius * 0.1;
g.stroke();

```

```

g.beginPath();          //inner circle (時鐘的中心點位置)

g.arc(0, 0, 10, 0, 2 * Math.PI);
g.fillStyle = '#fff';
g.fill();
}

```

```

//drawText();          //畫 clock 0~11 的數字, 共 12 格, 所以每格角度為 PI/6

```

```

function drawText() {
    g.font = 24 + "px arial";
    g.textBaseline = "middle";
    g.textAlign = "center";
    for (i = 0; i < 12; i++) {
        g.save();
        g.rotate(Math.PI / 6 * i);
        g.translate(0, -radius * 0.85);
        g.rotate(-Math.PI / 6 * i);
        g.fillText(" " + i, 0, 0);
        g.restore();
    }
}

```

```

//drawTime();

```

```

function drawTime() {
    var date = new Date();

```

畫 clock 的秒針、分針、時針

```

seconds = date.getSeconds() * 6;    // 360/60

```

```

minites = (date.getMinutes() + date.getSeconds() / 60) * 6;    // 360/60

```

```

hours = (date.getHours() + (date.getMinutes() + date.getSeconds() / 60) / 60) % 12 * 30;

```

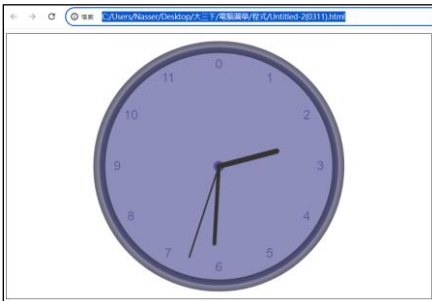
//將 minites 除 60 · 轉為小時 ; 轉 12 小時制 · 每小時 30 度

```

drawHand(seconds, radius * 0.8, 3);
drawHand(minites, radius * 0.65, 7);
drawHand(hours, radius * 0.5, 10);
}
//drawHand();
function drawHand(angle, length, width) {
    g.save();
    g.beginPath();
    g.lineCap = 'round';
    g.lineWidth = width;
    g.rotate(angle * Math.PI / 180);
    g.moveTo(0, 0);
    g.lineTo(0, -length);
    g.stroke();
    g.restore();
}
</script>
</body>

</html>

```



2. Recursive for generating Curves_a

```

<html>

<body>
    <canvas id="mycanvas" width="800" height="500" style="border:1px solid ;"></canvas>
    <script>
        var canvas = document.getElementById("mycanvas");
        var ctx = canvas.getContext('2d');

        drawCircle(320, 180, 200);           //產生 2d 畫布及圓

        //ctx.strokeRect(0, 0, 640, 360);

        function drawCircle(x, y, r) {

```

```

ctx.beginPath();
ctx.arc(x, y, r, 0, 2 * Math.PI);    //中間的第一顆大圓
ctx.stroke();

ctx.beginPath();
ctx.arc(x - r, y, r / 2, 0, 2 * Math.PI);

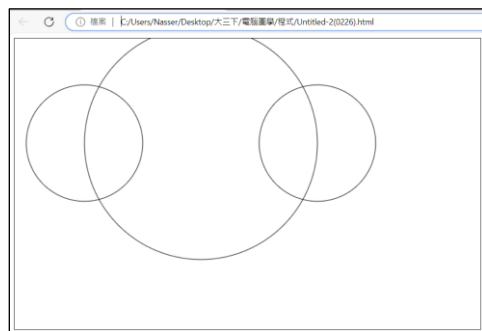
//從中間的大圓(r/2) · 往左邊位置(x - r) · 畫出小圓

ctx.stroke();

ctx.beginPath();
ctx.arc(x + r, y, r / 2, 0, 2 * Math.PI);
ctx.stroke();

}    //從中間的大圓(r/2) · 往右邊位置(x + r) · 畫出小圓
</script>
</body>
</html>

```



3. Recursive for generating Curves_ b

```

<html>

<body>
  <canvas id="mycanvas" width="800" height="500" style="border:1px solid ;"></canvas>
  <script>
    var canvas = document.getElementById("mycanvas");
    var ctx = canvas.getContext('2d');
    drawCircle(320, 180, 200);
    //ctx.strokeRect(0, 0, 640, 360);

```

```
function drawCircle(x, y, r) {
    ctx.beginPath();
    ctx.arc(x, y, r, 0, 2 * Math.PI);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(x - r, y, r / 2, 0, 2 * Math.PI);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(x + r, y, r / 2, 0, 2 * Math.PI);
    ctx.stroke();
}
```

```
function drawCircle(x, y, r) {
    ctx.beginPath();
    ctx.arc(x, y, r, 0, 2 * Math.PI);
    ctx.stroke();
    if (r > 2) {
        // condition for drawing similarity, 畫到最小的程度並且停止
        drawCircle(x + r, y, r / 2);
        drawCircle(x - r, y, r / 2);
    }
}
```

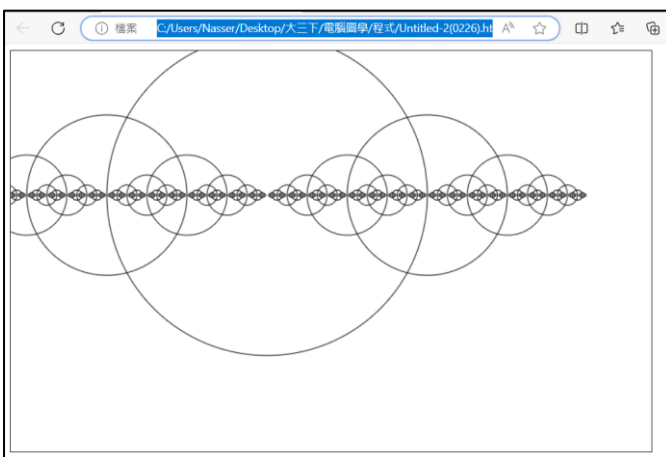
} //跟上面的想發類似，依序用圓的中心，接著下去繼續畫圓(利用遞迴方式)

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```



4. Recursive for generating Curves_c(add two branches)

<html>

<body>

<canvas id="mycanvas" width="640" height="360" style="border:1px solid ;"></canvas>

<script>

var canvas = document.getElementById("mycanvas");

var ctx = canvas.getContext('2d');

drawCircle(canvas.width/2, canvas.height/2, 200);

//ctx.strokeRect(0, 0, 640, 360);

```
function drawCircle(x, y, r) {
    ctx.beginPath();
    ctx.arc(x, y, r, 0, 2 * Math.PI); ctx.stroke();
    if (r > 2) { // condition for drawing similarity
        drawCircle(x + r, y, r / 2);
        drawCircle(x - r, y, r / 2);
        drawCircle(x, y+r, r / 2);
        drawCircle(x, y-r, r / 2);
    }
}
```

利用上述的方式(原本的為左右擴展(x))，現在加上上下方式(y)擴展圓的延伸，並利用遞迴方式完成(畫到最小的程度並且停止)

</script>

</body>

</html>



★心得：

這週講解的內容較多，需要較多時間來吸收與複習，特別是 Digital Clock，除了位置需要理解為何需要這樣寫之外，還利用之前所學的圓，圓外框的繪製、圓裡面 1~12 的角度配置(因為共有 12 格，因此每格是 $360/12=30(\text{PI}/30)$)，並且思考分針與秒針該如何繪製與移動(將分除 60，轉為小時；轉 12 小時制，每小時 30 度)。接著學習 Recursive 的繪畫，一步一步繪製圓與圓之間的關係(從中間的大圓($r/2$)，分別往左右繪畫，畫出小圓)，利用遞迴(recursive)的方式應用，畫到最小程度為

電腦圖學 Week 5-6

S1061103 許鉦偉

止，而最後自我的練習，除了課程所教的左右擴展之外，也上下擴展，利用遞迴方式，畫到不能再畫的程度，會呈現很密集的圓與類似八角形的圖案出現，是滿有趣的 coding 過程。總結來說，我認為較難的部分是時鐘的繪畫，應用之前所學的，並思考位置與角度等，至於遞迴的部分可以多在嘗試，說不定會有更多有趣的圖案。

●Week 6

1. 個人上台分享 Hw1 報告

My Hw1 報告展示

(修改煙火的顏色、重力加速度、增加煙火發射為隨機方向_改變角度、背景增加繪製星星圖案)

```
<canvas id="mycanvas" width="1200" height="600" style="border:1px solid ;"></canvas>
<html>
```

```
<body>
```

```
<script>
```

```
var canvas = document.getElementById("mycanvas");
var ctx = canvas.getContext("2d");
var colors = ["red", "green", "yellow", "blue", "purple", "white"];
class Particle {
  constructor(x, y, color) {
    this.x = x;
    this.y = y;
    this.color = colors[Math.floor(Math.random() * colors.length)];
    this.vx = (Math.random() - 0.5) * 20;
    this.vy = (Math.random() - 0.5) * 20;
    this.alpha = 1;
    this.friction = Math.random() * 0.05 + 0.9;
  }
  draw() {
    ctx.globalAlpha = this.alpha;
    ctx.beginPath();
    ctx.arc(this.x, this.y, 2, 0, Math.PI * 2);
    ctx.fillStyle = this.color;
    ctx.fill();
  }
  update() {
    this.vx *= this.friction;
    this.vy *= this.friction;
    this.x += this.vx;
    //v=v0+gt
    this.vy += 0.0255; //增加重力加速度(煙火火花往下墜的視覺效果)
    this.y += this.vy;
    this.alpha -= 0.01;
  }
}
```

```
class Firework {
  constructor(x, y, color) {
    this.x = x;
    this.y = y;
    //this.color = color; // 固定顏色
    this.color = colors[Math.floor(Math.random() * colors.length)];
    this.vx = 0;
    this.vy = Math.random() * -2 - 0.5;
    this.particles = [];
    this.lifespan = 180;
    this.hasExploded = false;
  }
  draw() {
    ctx.beginPath();
    ctx.arc(this.x, this.y, 3, 0, Math.PI * 2);
    ctx.fillStyle = this.color;
    ctx.fill();
  }

  explode() {
    for (var i = 0; i < 100; i++) {
      this.particles.push(new Particle(this.x, this.y, this.color));
      var angle = Math.random() * Math.PI * 2; // 隨機角度
      var speed = Math.random() * 2 + 1; // 隨機速度
      var vx = Math.cos(angle) * speed; // 計算水平速度
      var vy = Math.sin(angle) * speed; // 計算垂直速度
      this.particles.push(new Particle(this.x, this.y, this.color, vx, vy));
    }
  }
  update() {
    this.lifespan--;

    if (this.lifespan <= 0 && !this.hasExploded) {
      this.explode();
      this.vx = 0; this.vy = 0;
      this.hasExploded = true;
    } else if (this.lifespan > 0) {
      this.x += this.vx; // 添加水平速度以改變方向
      this.y += this.vy;
    }
  }
}
```

```
        for (var i = 0; i < this.particles.length; i++) {
            this.particles[i].update();
            this.particles[i].draw();
        }
    }
}

fireworks = [];
function drawStars() {
    for (let i = 0; i < 100; i++) {
        const x = Math.random() * canvas.width;
        const y = Math.random() * canvas.height;
        ctx.fillStyle = 'white';
        ctx.fillRect(x, y, 2,2);    // 2x2 像素的星星
    }
}

function animate() {

    ctx.fillStyle = 'rgba(0, 0, 0, 0.1)';
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    drawStars();    // 繪製星星

    for (var i = 0; i < fireworks.length; i++) {
        fireworks[i].update();
        fireworks[i].draw();
        if (fireworks[i].lifespan <= 0 && fireworks[i].particles[0].alpha <= 0) {
            //fireworks.splice(i, 1);
            fireworks[i].x = Math.random() * canvas.width;
            fireworks[i].y = 600;
            fireworks[i].color = 'rgb(' + Math.random() * 255 + ',' + Math.random() * 255
+ ',' + Math.random() * 255 + ')';
            fireworks[i].angle = Math.random() * Math.PI * 2; // 隨機角度
            fireworks[i].vx = Math.cos(fireworks[i].angle) * (Math.random() * 2 + 1);
            // 水平速度
            fireworks[i].vy = Math.sin(fireworks[i].angle) * (Math.random() * 2 + 1);
            // 垂直速度
            //fireworks[i].vx = 0;
        }
    }
}
```

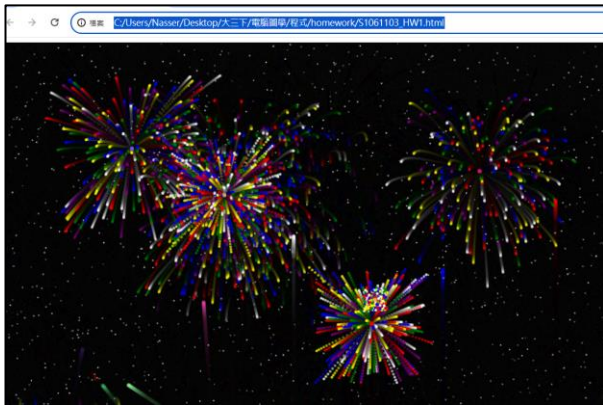
```

        //fireworks[i].vy = Math.random() * -2.5 - 0.5;
        fireworks[i].particles = [];
        fireworks[i].lifespan = 180;
        fireworks[i].hasExploded = false;
    }
};

if (Math.random() < 0.01) {
    const x = Math.random() * canvas.width;
    const color = 'rgb(' + Math.random() * 255 + ',' + Math.random() * 255 + ',' +
Math.random() * 255 + ')';
    fireworks.push(new Firework(x, canvas.height, color));
}
requestAnimationFrame(animate);
}
animate();
</script>
</body>

</html>

```



2. HTML 演練及講解

1. The Recursive Tree_a

```

<html>
<canvas id="fractal" width="1200" height="600" style="border:1px solid ;"></canvas>

<body>
    <style>
    </style>
    <script>

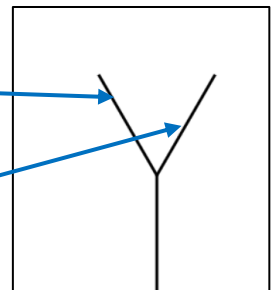
```

```
function line(x1, y1, x2, y2, width = 3) {
    ctx.strokeStyle = this.color;
    ctx.lineWidth = width;
    ctx.beginPath();
    ctx.moveTo(x1, y1);
    ctx.lineTo(x2, y2);
    ctx.closePath();
    ctx.stroke();
} //畫出線的颜色及粗細
```

```
var canvas = document.getElementById("fractal");
var ctx = canvas.getContext('2d');
```

```
var canvas = document.getElementById("fractal"); var ctx = canvas.getContext('2d');
var pos = {
    x: canvas.clientWidth / 2, y: canvas.clientHeight
};
```

```
ctx.translate(pos.x, pos.y); var len = 120;
var theta = 30; line(0, 0, 0, -len);
ctx.translate(0, -len); // move coordinate frame to the tree top
ctx.save();
ctx.rotate(Math.PI / 180 * theta); line(0, 0, 0, -len);
ctx.restore();
ctx.save();
this.ctx.rotate(-Math.PI / 180 * theta); line(0, 0, 0, -len);
ctx.restore();
```

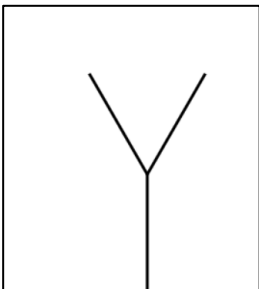


```
</script>
```

//分別儲存狀態，並畫上左右線條

```
</body>
```

```
</html>
```



2. The Recursive Tree_b

```
<html>
<canvas id="fractal" width="1200" height="600" style="border:1px solid ;"></canvas>

<body>
  <style>

  </style>
  <script>
    function line(x1, y1, x2, y2, width = 3) {
      ctx.strokeStyle = this.color;
      ctx.lineWidth = width;
      ctx.beginPath();
      ctx.moveTo(x1, y1);
      ctx.lineTo(x2, y2);
      ctx.closePath();
      ctx.stroke();
    }

    var canvas = document.getElementById("fractal");
    var ctx = canvas.getContext('2d');

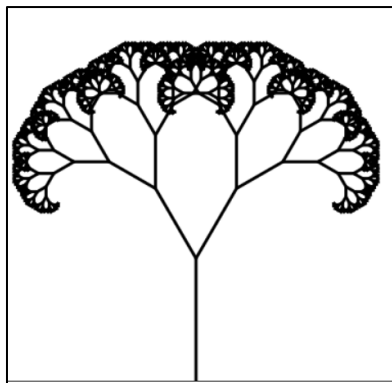
    ctx.translate(canvas.clientWidth / 2, canvas.clientHeight); //樹的位置

    var len = 120;
    var theta = 30;

    branch(len);
    function branch(len) {
      line(0, 0, 0, -len); ctx.translate(0, -len); len *= 0.66;
      if (len > 1) {
        // right branch
        ctx.save();
        ctx.rotate(Math.PI / 180 * theta); branch(len);
        ctx.restore();
        // left branch
        ctx.save();
        ctx.rotate(-Math.PI / 180 * theta); branch(len);
        ctx.restore();
      }
    }
  </script>
</body>
</html>
```

利用遞迴的方式，產生左右兩旁的樹枝，畫到最小的程度並且停止

```
    }  
  }  
</script>  
</body>  
  
</html>
```



★心得：

本周的近兩堂課，為報告 Hw1(Particle 或煙火的繪畫製作)，而我的部分除了指定修改煙火的顏色外，增加老師上課所講的重力加速度(this. vy += 0.0255;)，可以讓煙火呈現更真實的掉落畫面；增加水平與垂直的速度，可以讓煙火隨機在不同角落中發射；增加背景為會動的星空，使背景更加豐富鮮明，以上三大項目是我對這份作業更改的想法，特別是在可以隨機任意發射的角度部分。接著，則是接力上週所學的遞迴(recursive)，利用同樣的構想繪製出 recursive tree，跟上週圓的構想類似，但這次應用在樹枝上，往左右兩旁擴展樹枝(一樣是畫到最小程度才會停止)，讓樹枝呈現越來越茂密的圖案。

總結，本次課程印象深刻為看到許多同學對於 particle 的想法有很多不同的變化，每個人的構想與巧思都滿特別的，創意十足，印象中有 101 煙火的呈現、字幕發亮的動畫、星空的動畫、互動式 particle 的變化等等，這些技術與練習，能夠提升對於 html coding 的練習與想法，也看到每個人都在作業上做了很大的功夫，呈現不同面貌多元的巧思。

●Week 7

1. 簡單複習上次所學的 Recursive Tree

自我練習

```
<html>
<canvas id="fractal" width="1200" height="600" style="border:1px solid ;"></canvas>

<body>
  <style>

  </style>
  <script>
    function line(x1, y1, x2, y2, width = 3) {
      ctx.strokeStyle = this.color;
      ctx.lineWidth = width;
      ctx.beginPath();
      ctx.moveTo(x1, y1);
      ctx.lineTo(x2, y2);
      ctx.closePath();
      ctx.stroke();
    }

    var canvas = document.getElementById("fractal");
    var ctx = canvas.getContext('2d');

    ctx.translate(canvas.clientWidth / 2, canvas.clientHeight); //樹的位置

    var len = 120;
    var angles = [30, 45, 60, 90]; // Array of different angles

    branch(len, angles);

    function branch(len, angles) {
      for (var i = 0; i < angles.length; i++) {
        line(0, 0, 0, -len);
        ctx.translate(0, -len);
        len *= 0.66;

        if (len > 1) {
          ctx.save();
          ctx.rotate(Math.PI / 90 * angles[i]);

          branch(len, angles); // Recursively call branch with the same angles array
        }
      }
    }
  </script>
</body>
</html>
```

樹枝角度的變化

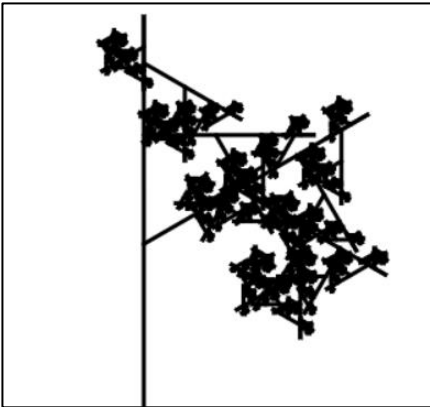
角度數字的不同，樹枝所
變化出樣子也不一樣

```

        ctx.restore();
    }
}
</script>
</body>

</html>

```



2. HTML 演練操作

1. Curve as Fractal

```

<html>
<canvas id="fractal" width="1200" height="600" style="border:1px solid;"></canvas>

<body>
  <style>

  </style>
  <script>
    var canvas = document.getElementById("fractal");
    var ctx = canvas.getContext('2d');

    class Vec2 {
      constructor(x = 0, y = 0) {
        this.x = x;
        this.y = y;
      }
      static add(v1, v2) {
        let x = v1.x + v2.x;
        let y = v1.y + v2.y;
        return new Vec2(x, y);
      }
    }
  </script>

```

//相加

```

    }
    static sub(v1, v2) { //相減
        let x = v1.x - v2.x;
        let y = v1.y - v2.y;
        return new Vec2(x, y);
    }
    static rotate(p, theta) { //旋轉
        let x = Math.cos(Math.PI / 180 * theta) * p.x - Math.sin(Math.PI / 180 * theta)
* p.y;
        let y = Math.sin(Math.PI / 180 * theta) * p.x + Math.cos(Math.PI / 180 * theta)
* p.y;
        return new Vec2(x, y); //兩點之間的線性內插
    }
    static lerp(start, end, scale) { // linear interpolation between two points
        let x = start.x + (end.x - start.x) * scale;
        let y = start.y + (end.y - start.y) * scale;
        return new Vec2(x, y);
    }
}

function line(x1, y1, x2, y2, width = 3) { //線的基本功能、粗細、顏色

    ctx.strokeStyle = this.color;
    ctx.lineWidth = width;
    ctx.beginPath();
    ctx.moveTo(x1, y1);
    ctx.lineTo(x2, y2);
    ctx.closePath();
    ctx.stroke();
}

function koch(A, B, order) { // V 逆時針旋轉 60 度，加上 C 後，得到中間的 E 點
    if (order == 1)
        line(A.x, A.y, B.x, B.y, 'red');
    else {
        let C = Vec2.lerp(A, B, 0.333);
        let D = Vec2.lerp(A, B, 0.667);
        let V = Vec2.sub(D, C);
        let E = Vec2.add(Vec2.rotate(V, -60), C);
        koch(A, C, order - 1);
    }
}

```

```

        koch(C, E, order - 1);
        koch(E, D, order - 1);
        koch(D, B, order - 1);
    }
}

```

```

let p1 = new Vec2(100, 100);
let p2 = new Vec2(200, 100);
let p3 = new Vec2(150, 200);

```

```

koch(p1, p2, 3);
koch(p2, p3, 3);
koch(p3, p1, 3);

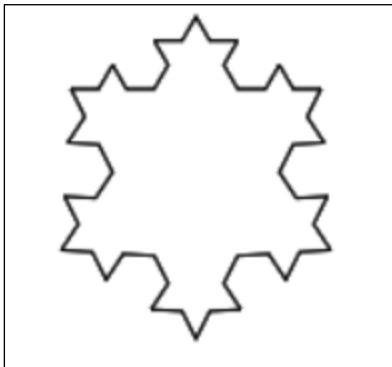
```

可以做調整、做變化(可以分裂成更小的三角形)

```

</script>
</body>
</html>

```



2. Curve as Fractal_subdivision

```

<html>
<canvas id="fractal" width="1200" height="600" style="border:1px solid;"></canvas>

<body>
    <style>

    </style>
    <script>
        var canvas = document.getElementById("fractal");
        var ctx = canvas.getContext('2d');

        class Vec2 {

```

```

    constructor(x = 0, y = 0) {
        this.x = x;
        this.y = y;
    }
    static add(v1, v2) {
        let x = v1.x + v2.x;
        let y = v1.y + v2.y;
        return new Vec2(x, y);
    }
    static sub(v1, v2) {
        let x = v1.x - v2.x;
        let y = v1.y - v2.y;
        return new Vec2(x, y);
    }
    static rotate(p, theta) {
        let x = Math.cos(Math.PI / 180 * theta) * p.x - Math.sin(Math.PI / 180 * theta)
* p.y;
        let y = Math.sin(Math.PI / 180 * theta) * p.x + Math.cos(Math.PI / 180 * theta)
* p.y;
        return new Vec2(x, y);
    }
    static lerp(start, end, scale) { // linear interpolation between two points
        let x = start.x + (end.x - start.x) * scale;
        let y = start.y + (end.y - start.y) * scale;
        return new Vec2(x, y);
    }
}

function line(x1, y1, x2, y2, width = 3) {

    ctx.strokeStyle = this.color;
    ctx.lineWidth = width;
    ctx.beginPath();
    ctx.moveTo(x1, y1);
    ctx.lineTo(x2, y2);
    ctx.closePath();
    ctx.stroke();
}

function triangle(Ax, Ay, Bx, By, Cx, Cy) {

```

```

line(Ax,Ay,Bx,By);
line(Bx,By,Cx,Cy);
line(Ax,Ay,Cx,Cy);

```

} 上面程式一樣不變

```

function subdivideTriangle(A, B, C, order) {
  if (order == 3) {
    triangle(A.x, A.y, B.x, B.y, C.x, C.y);
  }
  else {
    let AB = Vec2.lerp(A, B, 0.5);
    let BC = Vec2.lerp(B, C, 0.5);
    let AC = Vec2.lerp(A, C, 0.5);
    subdivideTriangle(A, AB, AC, order - 1);
    subdivideTriangle(AB, B, BC, order - 1);
    subdivideTriangle(AC, BC, C, order - 1);
  }
}

```

繪製三角形(ABC
三個點)

三角形三個邊，從中
間切割

```

let p1 = new Vec2(100, 100);
let p2 = new Vec2(200, 100);
let p3 = new Vec2(150, 200);

```

```

subdivideTriangle(p1, p2, p3, 5);

```

呼叫三角形三個點的座標

```

</script>

```

```

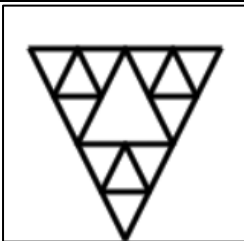
</body>

```

```

</html>

```



3. Pythagorean Tree

```
<html>  
<canvas id="fractal" width="1200" height="600" style="border:1px solid;"></canvas>
```

```
<body>
```

```
  <style>
```

```
</style>
```

```
  <script>
```

```
    var canvas = document.getElementById("fractal");
```

```
    var ctx = canvas.getContext('2d');
```

```
    var tree = {
```

樹的 x y
位置

```
      x: canvas.clientWidth / 2,
```

```
      y: canvas.clientHeight - 80,
```

```
      trunk: 80,
```

```
      30 度 langle: Math.PI / 6, // left angel of the tree trunk
```

```
      60 度 rangle: Math.PI / 3, // right angel of the tree trunk
```

```
      maxorder: 8, // generation
```

```
      colors: ['coral', 'green', 'seagreen', 'darkgreen', 'yellowgreen', 'darkblue']
```

```
    }
```

```
    ctx.save();
```

```
    ctx.fillStyle = tree.colors[0];
```

```
    ctx.translate(tree.x, tree.y);
```

```
    ctx.fillRect(0, 0, tree.trunk, tree.trunk);
```

```
    // draw left tree
```

```
    var lwidth = tree.trunk * Math.sin(tree.rangle);
```

```
    ctx.restore();
```

```
    ctx.save();
```

```
    ctx.rotate(-tree.langle);
```

```
    ctx.translate(0, -lwidth);
```

```
    ctx.fillStyle = tree.colors[1];
```

```
    ctx.fillRect(0, 0, lwidth, lwidth);
```

```
    ctx.restore();
```

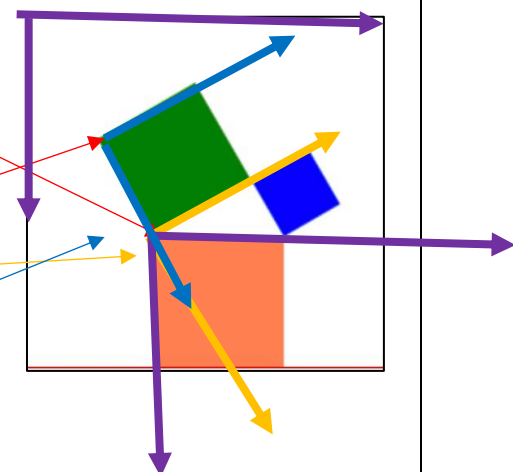
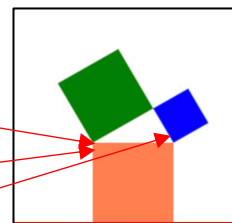
```
    // draw right tree
```

```
    var rwidth = tree.trunk * Math.sin(tree.langle);
```

原點

30 度

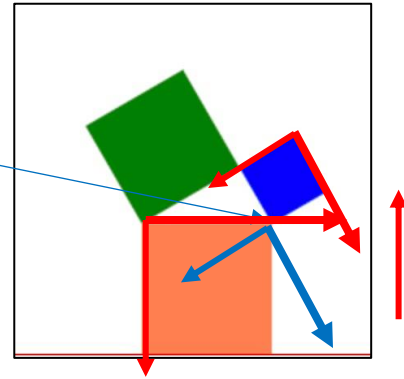
60 度



```

ctx.save();
ctx.translate(tree.trunk, 0);
ctx.rotate(tree.rangle);
ctx.translate(-rwidth, -rwidth);
ctx.fillStyle = 'blue';
ctx.fillRect(0, 0, rwidth, rwidth);
ctx.restore();

```



```

function branch(order, width) {
  if (order > tree.maxorder) {
    return;
  }
  // draw left tree
  var lwidth = width * Math.sin(tree.rangle);
  ctx.save();
  ctx.rotate(-tree.langle);
  ctx.translate(0, -lwidth);
  ctx.fillStyle = tree.colors[order];
  ctx.fillRect(0, 0, lwidth, lwidth);
  branch(order + 1, lwidth);
  ctx.restore();

```

Call 下層左樹

```

  // draw right tree
  var rwidth = width * Math.sin(tree.langle);
  ctx.save();
  ctx.translate(width, 0);
  ctx.rotate(tree.rangle);
  ctx.translate(-rwidth, -rwidth);
  ctx.fillStyle = 'blue';
  ctx.fillRect(0, 0, rwidth, rwidth);
  branch(order + 1, rwidth);
  ctx.restore();
}

```

Call 下層右樹

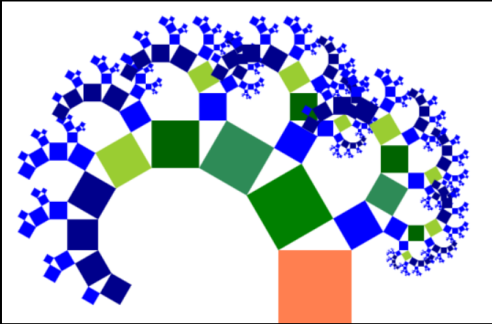
```

ctx.save();
ctx.fillStyle = tree.colors[0];
ctx.translate(tree.x, tree.y);
ctx.fillRect(0, 0, tree.trunk, tree.trunk);
branch(1, tree.trunk);
ctx.restore();

```

畫出第一個樹枝，之後利用
branch 遞迴畫出後續的樹枝

```
</script>  
</body>  
</html>
```



3. 講解 3D 概念

a. 介紹 3D 模型網站

a_1. Car visualizer

<https://carvisualizer.plus360degrees.com/threejs/>



a_2. moments of happiness

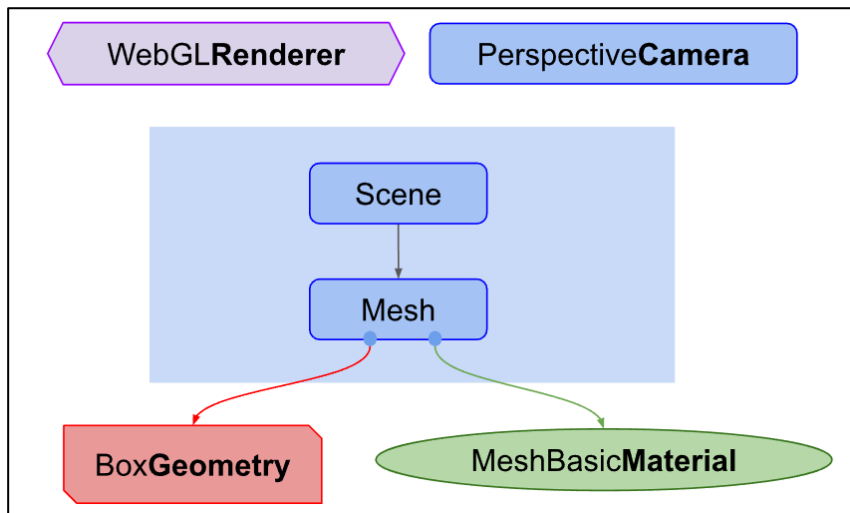
<https://moments.epic.net/#home>



b.

Three.js is a 3D library that tries to make it as easy as possible to get 3D content on a webpage. WebGL generally requires quite a bit of code and that is where three.js comes in.

c. Renderer(Scene, Mesh, Camera, Geometry, Material)



★心得：

本週課程除了先複習上次所學的 recursive tree 之外，接著額外提到樹枝的角度可以改變，會形成不同的樣式，而這週上了碎形的圖案與切割，碎形的部分還尚可以理解，能快速懂得每行程式的意義，花較多時間講解的則是 Pythagorean Tree，圖形較為複雜，須用畫圖理解每個程式執行的動作(ex:旋轉、上下移動等等)，因此在我的筆記解釋上也用畫圖的方式來解釋，方便日後複習或參考時，能夠快速理解程式的意思。最後，花一些時間介紹 3D 的概念，跟之前所學的 Unity 或 Maya、Blender 有些類似，因此較好理解，懂得老師上課所敘述的部分。

電腦圖學 Week 7-8
S1061103 許鉦偉

● Week 8

校際交流日放假(4/8-9)

●Week 9

期中考週一停課

●Week 10

1. 複習上堂課所教的 3D 模型概念

2. 解釋程式模型架構

const renderer;	var → 函數
const scene;	let → 區塊
const camera;	const → 區塊

骨架表示	
<pre><script type="module"> import * as THREE from ...; </script></pre>	<pre>type="module"</pre> enables us to use the import keyword to load three.js
<pre>import * as THREE from "https://cdn.skypack.dev/three@0.132.2"</pre>	import everything from the three.js core into main.js under the THREE namespace(將 3D 模型架構導入程式裡)
<pre>import { OrbitControls } from "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";</pre>	importing plugins (a camera controls plugin)(導入 3D 模型相機)

3. 操作演練，如何建構簡單的 3D cube

1_a. 課堂練習之 cube_step1(建立平面方塊)
<pre><!DOCTYPE html> <html> <head> <title>My first three.js app</title> <style> body {</pre>

```

margin: 0;
}
</style>
</head>

<body>
  <canvas id="c"></canvas>
  <script type="module">

```

```

import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
import { OrbitControls } from
"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

```

3D 模型的基礎骨架

```

const canvas = document.getElementById('c');
const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
const scene = new THREE.Scene();

```

建立 3D 模型的畫布

```

const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshBasicMaterial({ color: 0x00FF00 });
const mesh = new THREE.Mesh(geometry, material);

```

```
scene.add(mesh);
```

建立 Cube 方塊
(基本的=長*寬*高)
(顏色及材質)

```

const fov = 75;
const aspect = 2; // the canvas default
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
camera.position.z = 2;

```

```
renderer.render(scene, camera);
```

fov = 觀看模型的視野，這裡設為 75 度

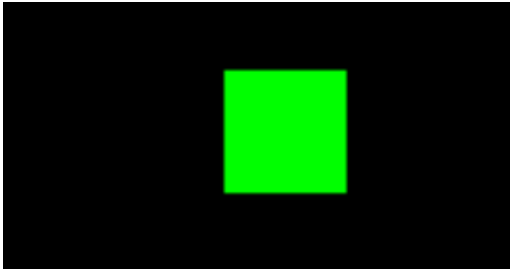
aspect = 畫布比例，預設通常為 2 (300*150, 300/150)

```

</script>
</body>

</html>

```



1-b. 課堂練習之 cube_step2(建構出基本的 3D Cube 模型)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My first three.js app</title>
```

```
  <style>
```

```
    body { margin: 0; }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <canvas id="c"></canvas>
```

```
  <script type="module">
```

```
    import * as THREE from
```

```
"https://cdn.skypack.dev/three@0.132.2";
```

```
    import { OrbitControls } from
```

```
"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";
```

```
    const canvas = document.getElementById('c');
```

```
    const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
```

```
    const fov = 75;
```

```
    const aspect = 2; // the canvas default
```

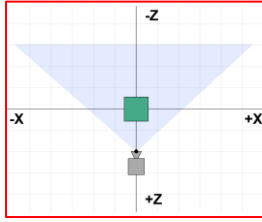
```
    const near = 0.1;
```

```
    const far = 5;
```

```
    const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
```

3D 模型的基礎骨架，
建立 3D 模型的畫布

建立 Cube 方塊
(基本的=長*寬*高)
(顏色及材質)

camera.position.z = 2;

fov = 觀看模型的視野，這裡設為 75 度

aspect = 畫布比例，預設通常為 2 (300*150, 300/150)

camera.position.z=2 表示鏡頭位置

(通長鏡頭位置在+z 上，照射範圍則是-z 的方向)

```
const scene = new THREE.Scene(); //把建置的 3D 內容，加入場景中
const boxWidth = 1; //寬
const boxHeight = 1; //高
const boxDepth = 1; //長
const geometry = new THREE.BoxGeometry(boxWidth, boxHeight, boxDepth);
//箱子指令
```

```
const material = new THREE.MeshBasicMaterial({ color: 0x00FF00 }); //材質 顏色
const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);
```

Mesh（網格）是由以下三個要素組成的：

Geometry（幾何形狀）：物體的形狀。

Material（材質）：如何繪製物體，包括光澤程度、顏色、應用的紋理等。
物體的位置、方向和比例。

```
const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);
```

燈光:

燈光的顏色

燈光的強度

燈光的位置

把燈光加入置場景中

```
function render(time) {
    time *= 0.001; // convert time to seconds

    mesh.rotation.x = time; //x 軸旋轉
    mesh.rotation.y = time; //y 軸旋轉

    renderer.render(scene, camera);

    requestAnimationFrame(render);
}
```

cube 的移動:

旋轉，並能立即更新

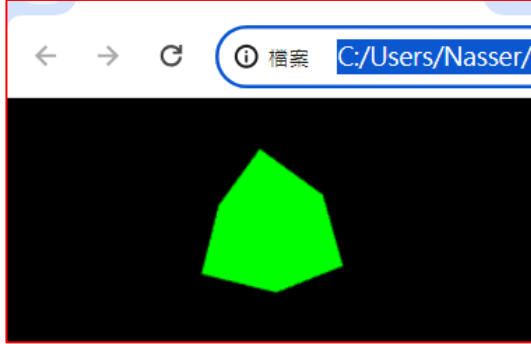
(scale 放大/縮小

rotation 旋轉)

```

    }
    requestAnimationFrame(render);
  </script>
</body>
</html>

```



1_c. 課堂練習之 cube_step3(增加可以控制 ex:放大/縮小/點擊移動)

```

<!DOCTYPE html>
<html>

<head>
  <title>My first three.js app</title>
  <style>
    body {
      margin: 0;
    }
  </style>
</head>

<body>
  <canvas id="c" width="500" height="500"></canvas>
  <script type="module">
    import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
    import { OrbitControls } from
    "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

    const canvas = document.getElementById('c');
    const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
    const scene = new THREE.Scene();
    const geometry = new THREE.BoxGeometry(1, 1, 1);
    const material = new THREE.MeshPhongMaterial({ color: 0x00FF00 });
    //材質改為能夠被光反射的材質(與上述材質不一樣)

```

```
const mesh = new THREE.Mesh(geometry, material);
mesh.rotation.x = 60; //增加其旋轉的大小
mesh.rotation.y = 50;
//mesh.scale.x=2;

scene.add(mesh);

const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight; //畫布比例的大小呈現
//const aspect = 2; // the canvas default
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement);
camera.position.z = 2;
//controls.update();

const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);

renderer.render(scene, camera);

function render(time) {
    time *= 0.001; // convert time to seconds

    mesh.rotation.x = time;
    mesh.rotation.y = time;

    controls.update();

    renderer.render(scene, camera);

    requestAnimationFrame(render);
}
requestAnimationFrame(render);
```

增加控制的指令(相機)，可縮放
並移動

燈光位置

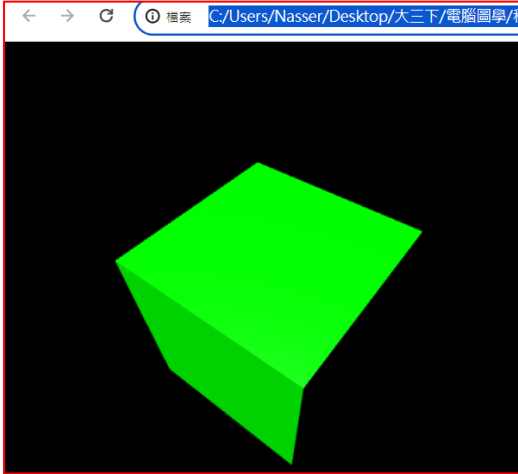
旋轉

```

</script>
</body>

</html>

```



2. 自我練習 Add 2 more cubes: different material 、different position and rotating angle

```

<!DOCTYPE html>
<html>

<head>
  <title>My first three.js app</title>
  <style>
    body {
      margin: 0;
    }
  </style>
</head>

<body>
  <canvas id="c" width="500" height="500"></canvas>
  <script type="module">
    import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
    import { OrbitControls } from
    "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

    const canvas = document.getElementById('c');
    const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
    const scene = new THREE.Scene();

```

```
const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshPhongMaterial({ color: 0x00FF00 });
const mesh = new THREE.Mesh(geometry, material);
mesh.rotation.x = 60;
mesh.rotation.y = 50;
scene.add(mesh);
```

方塊 2 的設定

```
const material2 = new THREE.MeshPhongMaterial({ color: 0xFF0000 });
const geometry2 = new THREE.BoxGeometry(1, 1, 1);
const mesh2 = new THREE.Mesh(geometry2, material2);
mesh2.position.x = -2;
mesh2.rotation.x = -45;
mesh2.rotation.y = 30;
scene.add(mesh2);
```

```
const material3 = new THREE.MeshPhongMaterial({ color: 0x0000FF });
const geometry3 = new THREE.BoxGeometry(1, 1, 1);
const mesh3 = new THREE.Mesh(geometry3, material3);
mesh3.position.x = 2;
mesh3.rotation.x = 30;
mesh3.rotation.y = -45;
scene.add(mesh3);
```

方塊 3 的設定

```
const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight;
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement);
camera.position.z = 5;

const color = 0xFFFFFF;
const intensity = 3;
const light = new
THREE.DirectionalLight(color,
intensity);
light.position.set(-1, 2, 4);
scene.add(light);
```

fov = 觀看模型的視野，這裡設為 75 度

aspect = 畫布比例，預設通常為 2 (300*150, 300/150)

camera.position.z=2 表示鏡頭位置

(通長鏡頭位置在+z 上，照射範圍則是-z 的方向)

燈光設定

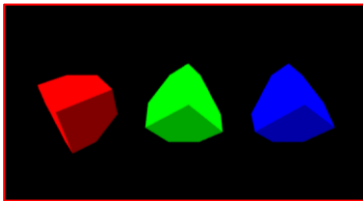
```
function render(time) {  
    time *= 0.001; // convert time to seconds  
  
    mesh.rotation.x = time;  
    mesh.rotation.y = time;  
    mesh2.rotation.x = -time;  
    mesh2.rotation.y = -time;  
    mesh3.rotation.x = time;  
    mesh3.rotation.y = -time;  
  
    controls.update(); // Update controls here  
  
    renderer.render(scene, camera);  
  
    requestAnimationFrame(render);  
}  
requestAnimationFrame(render);
```

方塊 1~3 的旋轉

</script>

</body>

</html>



★心得

隔了兩週回到課程，為了銜接上次所教的內容，先複習一下之前所提到的 3D 基本架構，接著才繼續講解這禮拜的課程重點。

首先，免不了的一定是需要知道整體 3D 模型在 script 前的基本架構，包含如何建構出畫布(跟 2d 一樣的概念)，再來才開始畫簡單的 3D cube。我認為因為先前已有學過 2D 的架構，因此在繪畫 3D 模型的時候，反倒不會這麼難，只需要了解簡單的幾個基本指令與意思。

總結來說，這週的課程內容，並不覺得很難或非常複雜，而是很快地可以進入狀況，來做演練，特別的是最後所學了指令(control)很方便，也很常應用到。在接下來的課程中，應該還是會繼續使用到這幾個今天所教的基本功能，來繼續做延伸學習。

●Week 11

※延續上次的程式(正方體、圓體等)，加入變化

1. 操作演練

(1) Responsive Design 響應式設計_a(視窗拉動“影響”物體改變 ex:扁的)

`<!DOCTYPE html>``<html>``<head>``<title>My first three.js app</title>``<style>``html,``body {``margin: 0;``height: 100%;``}``#c {``width: 100%;``height: 100%;``display: block;``}``</style>``</head>``<body>``<canvas id="c" width="500" height="500"></canvas>``<script type="module">``import * as THREE from "https://cdn.skypack.dev/three@0.132.2";``import { OrbitControls } from``"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";``const canvas = document.getElementById('c');``const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });``const scene = new THREE.Scene();``const geometry = new THREE.BoxGeometry(1, 1, 1);``const material = new THREE.MeshPhongMaterial({ color: 0x00FF00 });``const mesh = new THREE.Mesh(geometry, material);``mesh.rotation.x = 60;``mesh.rotation.y = 50;`

網頁在不同大小的顯示器上（從桌面電腦到平板電腦到手機）都能正常顯示。

將 HTML 和 body 的高度設置為 100% 使它們填滿整個視窗。

將其顯示模式設置為 block。(新增第 1 個)

```
scene.add(mesh);

const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight;
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement);
camera.position.z = 2;

const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);

function render(time) {
    time *= 0.001; // convert time to seconds

    mesh.rotation.x = time;
    mesh.rotation.y = time;

    controls.update(); // Update controls here

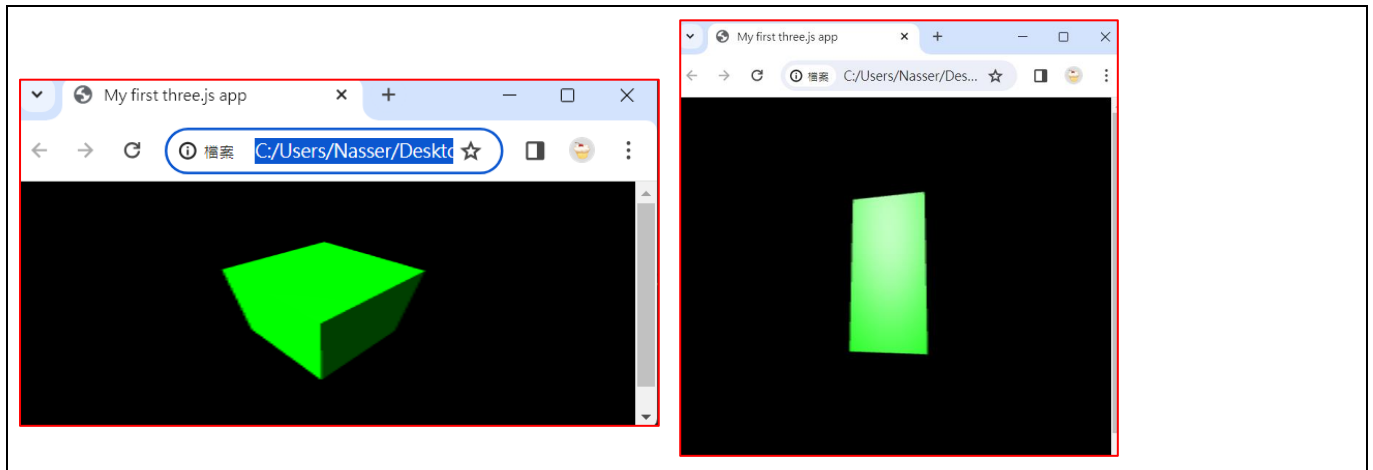
    renderer.render(scene, camera);

    requestAnimationFrame(render);
}
requestAnimationFrame(render);
```

</script>

</body>

</html>



(2) Responsive Design 響應式設計_b(視窗拉動“不影響”物體改變，按比例縮放)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>My first three.js app</title>
```

```
<style>
```

```
html,
```

```
body {
```

```
margin: 0;
```

```
height: 100%;
```

```
}
```

```
#c {
```

```
width: 100%;
```

```
height: 100%;
```

```
display: block;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<canvas id="c" width="250" height="500"></canvas>
```

```
<script type="module">
```

```
import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
```

```
import { OrbitControls } from
```

```
"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";
```

網頁在不同大小的顯示器上（從桌面電腦到平板電腦到手機）都能正常顯示。

將 HTML 和 body 的高度設置為 100% 使它們填滿整個視窗。

將其顯示模式設置為 block。(新增第 1 個)

```
const canvas = document.getElementById('c');
const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
const scene = new THREE.Scene();

const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshPhongMaterial({ color: 0x00FF00 });
const mesh = new THREE.Mesh(geometry, material);
mesh.rotation.x = 60;
mesh.rotation.y = 50;

scene.add(mesh);

const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight;
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement);
camera.position.z = 2;

const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);
```

```
function resizeRendererToDisplaySize(renderer) {
    const needResize =
        (canvas.width !== canvas.clientWidth) ||
        (canvas.height !== canvas.clientHeight);

    if (needResize) {
        renderer.setSize(canvas.clientWidth,
            canvas.clientHeight, false);
    }
    return needResize;
}
```

Canvas 元素有兩個尺寸。一個尺寸是在頁面上顯示的 canvas 尺寸。另一個尺寸是 canvas 內部實際的像素數量。利用 `renderer.setSize` 設置 canvas 的內部尺寸。將 `false` 設置為 canvas 的大小不會與其他元素有所不同。

1. `render` 的大小跟畫布有沒有一樣大(檢測)
2. 重設 `render` 的大小跟畫布一樣(新增第 2 個)

```
function render(time) {
```

```
time *= 0.001; // convert time to seconds
```

```
mesh.rotation.x = time;
```

```
mesh.rotation.y = time;
```

```
controls.update(); // Update controls here
```

```
camera.aspect = canvas.clientWidth / canvas.clientHeight;
```

```
camera.updateProjectionMatrix();
```

```
renderer.render(scene, camera);
```

```
requestAnimationFrame(render);
```

```
}
```

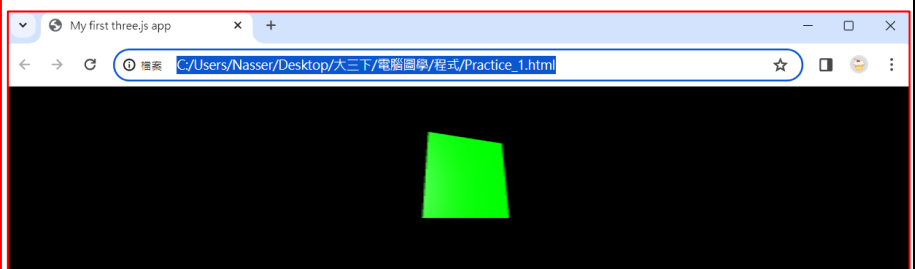
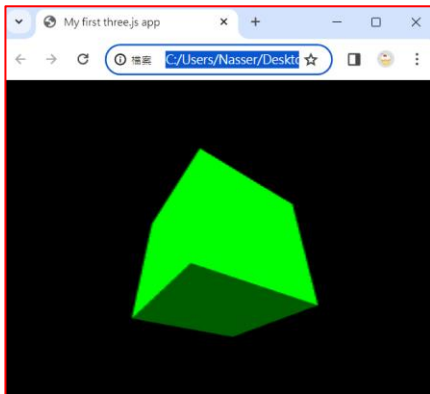
```
requestAnimationFrame(render);
```

```
</script>
```

```
</body>
```

```
</html>
```

當你改變了相機的屬性（如視角、畫面寬高比等）後，需要呼叫 `updateProjectionMatrix()` 來更新相機的投影矩陣，以使這些改變生效。這樣可以確保相機投影的正確性，使場景中的物體能夠按照預期的方式被投影到螢幕上。(新增第 3 個)



(3) SphereGeometry(圓體)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My first three.js app</title>
```

```
  <style>
```

```
html,
body {
  margin: 0;
  height: 100%;
}

#c {
  width: 100%;
  height: 100%;
  display: block;
}

</style>
</head>

<body>
  <canvas id="c" width="500" height="500"></canvas>
  <script type="module">
    import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
    import { OrbitControls } from
    "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

    const canvas = document.getElementById('c');
    const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
    const scene = new THREE.Scene();

    const geometry = new THREE.SphereGeometry(1, 50, 50);
    const material = new THREE.MeshPhongMaterial({ color: 0x00FF00 });
    const mesh = new THREE.Mesh(geometry, material);
    mesh.rotation.x = 60;
    mesh.rotation.y = 50;

    scene.add(mesh);

    const fov = 75;
    const aspect = canvas.clientWidth / canvas.clientHeight;
    const near = 0.1;
    const far = 5;
    const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
    const controls = new OrbitControls(camera, renderer.domElement);
```

(半徑,橫線數量(經度),縱線數量(緯度))

```
camera.position.z = 2;

const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);

function resizeRendererToDisplaySize(renderer) {
    const needResize =
        (canvas.width !== canvas.clientWidth) ||
        (canvas.height !== canvas.clientHeight);

    if (needResize) {
        renderer.setSize(canvas.clientWidth,
            canvas.clientHeight, false);
    }
    return needResize;
}

function render(time) {
    time *= 0.001; // convert time to seconds

    mesh.rotation.x = time;
    mesh.rotation.y = time;

    controls.update(); // Update controls here

    camera.aspect = canvas.clientWidth / canvas.clientHeight;

    camera.updateProjectionMatrix();

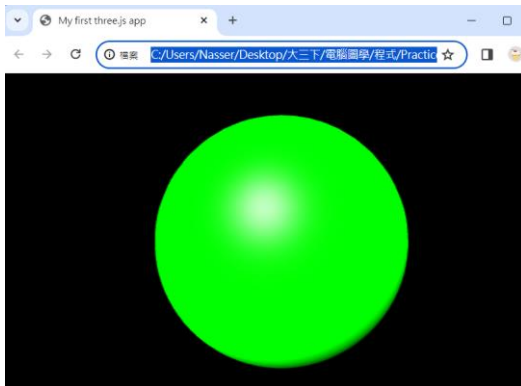
    renderer.render(scene, camera);

    requestAnimationFrame(render);
}
requestAnimationFrame(render);
```

</script>

</body>

</html>



(4) WireframeGeometry

<!DOCTYPE html>

<html>

<head>

<title>My first three.js app</title>

<style>

html,

body {

margin: 0;

height: 100%;

}

#c {

width: 100%;

height: 100%;

display: block;

}

</style>

</head>

<body>

<canvas id="c" width="500" height="500"></canvas>

<script type="module">

import * as THREE from "https://cdn.skypack.dev/three@0.132.2";

```
import { OrbitControls } from
"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

const canvas = document.getElementById('c');
const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
const scene = new THREE.Scene();

const geometry = new THREE.SphereGeometry(1, 50, 50);

//const wgeometry = new THREE.WireframeGeometry(sphereGeometry);

const material = new THREE.MeshBasicMaterial({ color: 0xffff00, wireframe: true });

加入可顯示線框的程式 Wireframe

//const material = new THREE.MeshPhongMaterial({ color: 0x00FF00 });

const mesh = new THREE.Mesh(geometry, material);
mesh.rotation.x = 60;
mesh.rotation.y = 50;

scene.add(mesh);

const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight;
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement);
camera.position.z = 2;

const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);

function resizeRendererToDisplaySize(renderer) {
    const needResize =
        (canvas.width !== canvas.clientWidth) ||
```

```

(canvas.height !== canvas.clientHeight);

if (needResize) {
    renderer.setSize(canvas.clientWidth,
        canvas.clientHeight, false);
}
return needResize;
}

```

```

function render(time) {
    time *= 0.001; // convert time to seconds

```

```

    mesh.rotation.x = time;

```

```

    mesh.rotation.y = time;

```

```

    controls.update(); // Update controls here

```

```

    camera.aspect = canvas.clientWidth / canvas.clientHeight;

```

```

    camera.updateProjectionMatrix();

```

```

    renderer.render(scene, camera);

```

```

    requestAnimationFrame(render);

```

```

}
requestAnimationFrame(render);

```

```

</script>

```

```

</body>

```

```

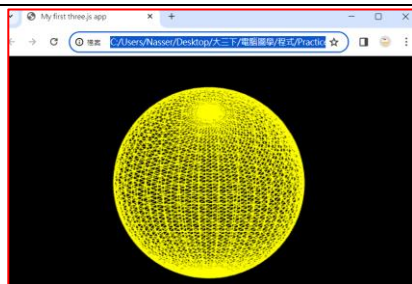
</html>

```

Canvas 元素有兩個尺寸。一個尺寸是在頁面上顯示的 canvas 尺寸。另一個尺寸是 canvas 內部實際的像素數量。利用 `renderer.setSize` 設置 canvas 的內部尺寸。將 `false` 設置為 canvas 的大小不會與其他元素有所不同。

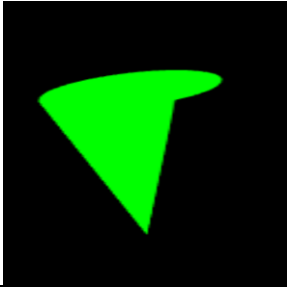
1. `render` 的大小跟畫布有沒有一樣大(檢測)
2. 重設 `render` 的大小跟畫布一樣 (新增第 2 個)

當你改變了相機的屬性（如視角、畫面寬高比等）後，需要呼叫 `updateProjectionMatrix()` 來更新相機的投影矩陣，以使這些改變生效。這樣可以確保相機投影的正確性，使場景中的物體能夠按照預期的方式被投影到螢幕上。(新增第 3 個)



(5) 自我練習_Cone(程式碼與上述一樣，只顯示修改部分)

```
const geometry = new THREE.ConeGeometry(0.5, 1, 50);
```



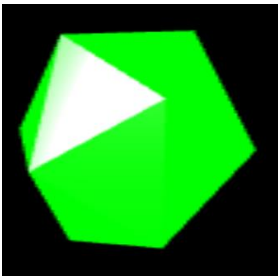
(6) 自我練習_Cylinder(程式碼與上述一樣，只顯示修改部分)

```
const geometry = new THREE.CylinderGeometry(0.5, 0.5, 1, 50);
```



(7) 自我練習_Icosahedron(二十面體)(程式碼與上述一樣，只顯示修改部分)

```
const geometry = new THREE.IcosahedronGeometry( 0.5 );
```



(8) 自我練習_Torus(環面體或甜甜圈)(程式碼與上述一樣，只顯示修改部分)

```
const geometry = new THREE.TorusGeometry(0.5, 0.2, 16, 100 );
```



(9) 自我練習_TorusKnot(環面結體)(程式碼與上述一樣，只顯示修改部分)

const geometry = new THREE.TorusKnotGeometry(0.5, 0.2, 150, 8,);



2. CH5_MATRIX 矩陣

(1) 矩陣表示

$$A_{n \times m} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1j} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2j} & \dots & a_{2m} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ a_{i1} & \dots & \dots & a_{ij} & \dots & a_{im} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nj} & \dots & a_{nm} \end{bmatrix}$$

Column j

Row i

(2) 計算方式

$$A_{n \times m} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1j} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2j} & \dots & a_{2m} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ a_{i1} & \dots & \dots & a_{ij} & \dots & a_{im} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nj} & \dots & a_{nm} \end{bmatrix}$$

Column j

Row i

element wise

$$B_{n \times m} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1j} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2j} & \dots & b_{2m} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ b_{i1} & \dots & \dots & b_{ij} & \dots & b_{im} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nj} & \dots & b_{nm} \end{bmatrix}$$

$$C_{n \times m} = A_{n \times m} + B_{n \times m} \quad C_{n \times m} = A_{n \times m} - B_{n \times m}$$

$$C_{ij} = a_{ij} + b_{ij} \quad C_{ij} = a_{ij} - b_{ij}$$

$$\begin{bmatrix} 2 & 1 \\ 3 & 5 \end{bmatrix} + \begin{bmatrix} 6 & 2 \\ 1 & 5 \end{bmatrix} = \begin{bmatrix} 8 & 7 \\ 4 & 6 \end{bmatrix}$$

$$A_{n \times m} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1j} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2j} & \dots & a_{2m} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ a_{i1} & \dots & \dots & a_{ij} & \dots & a_{im} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nj} & \dots & a_{nm} \end{bmatrix}$$

Column j

Row i

$$B_{m \times p} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1j} & \dots & b_{1p} \\ b_{21} & b_{22} & \dots & b_{2j} & \dots & b_{2p} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ b_{j1} & \dots & \dots & b_{jj} & \dots & b_{jp} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots \\ b_{m1} & b_{m2} & \dots & b_{mj} & \dots & b_{mp} \end{bmatrix}$$

Matrix Multiplication: Product

$$C_{n \times p} = A_{n \times m} B_{m \times p} \quad C_{ij} = a_{i \cdot} \cdot b_{\cdot j} = \sum_{k=1}^{k=m} a_{ik} \times b_{kj}$$

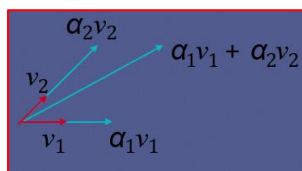
$$\begin{bmatrix} 2 & 1 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} 6 & 2 \\ 1 & 5 \end{bmatrix} = \begin{bmatrix} 2 \times 6 + 1 \times 1 & 2 \times 2 + 1 \times 5 \\ 3 \times 6 + 5 \times 1 & 3 \times 2 + 5 \times 5 \end{bmatrix} = \begin{bmatrix} 13 & 9 \\ 23 & 29 \end{bmatrix}$$

(3) Vector Space 向量空間

由一組線性獨立 (linear independent) 的向量稱為基底 $\{v_1, v_2, \dots, v_n\}$ 所構成。

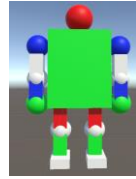
可經由向量加法(vector addition)與純量乘法(Scale multiplication)所擴展而成的線性空間。

$$v = \alpha_1 v_1 + \alpha_2 v_2$$



3. 公告 Hw2

- 1.請使用 geometry 相關指令(如 BoxGeometry, ConeGeometry, SphereGeometry, CylinderGeometry, TorusGeometry 等)，組合與建立一個簡單的三維機器人模型
- 2.不使用外部模型(obj 檔)或 maya、blender 等建模軟體
- 3.建立一個點光源(PointLight)，圍繞著機器人旋轉的動畫



4. 燈光轉動練習

燈光轉動練習_光源(物體代替)

```

<!DOCTYPE html>
<html>
<head>
  <title>My first three.js app</title>
  <style>
    html,
    body {
      margin: 0;
      /*不留白邊*/
      height: 100%;
    }

    #c {
      /*畫布的意思*/
      width: 100%;
      height: 100%;
      display: block;
      /*block 那個不寫的話，會多一個卷軸(因為會預設多一個東西(?*/
    }
  </style>
</head>

<body>
  <canvas id="c" width="1500" height="500"></canvas> <!--上面的設定其實會把這裡長寬吃掉，但也代表解析度(?-->
  <script type="module">
    import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
    import { OrbitControls } from
    "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

    const canvas = document.getElementById('c');
  </script>
</body>

```

```
const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
//scene
const scene = new THREE.Scene();

//物體 1
const geometry = new THREE.BoxGeometry(0.7, 0.7, 0.7);
//const material = new THREE.MeshBasicMaterial({ color: 0x00FF00 });
const material = new THREE.MeshPhongMaterial({ color: "pink" });
const cube = new THREE.Mesh(geometry, material);
//cube.scale.x = 0.5;

//物體 2
const sphereGeometry = new THREE.SphereGeometry(0.5, 20, 20);
const geometry2 = new THREE.WireframeGeometry(sphereGeometry);
const material2 = new THREE.MeshBasicMaterial({ color: 0xffff00, wireframe: true });
const mesh = new THREE.Mesh(geometry2, material2);
mesh.position.x = 2;

var group = new THREE.Group();
cube.add(mesh);

scene.add(cube);
// scene.add(group);

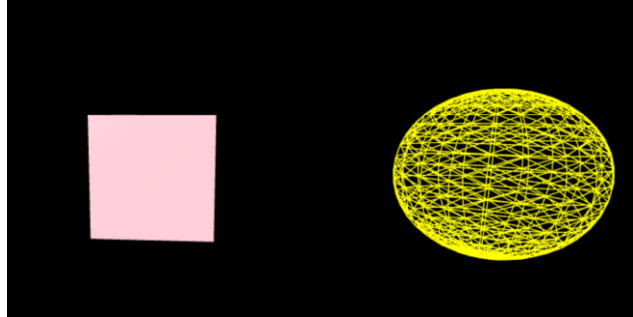
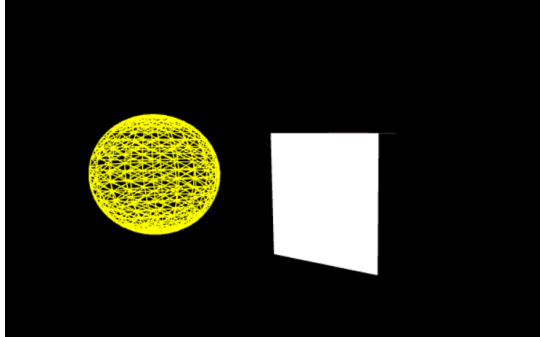
//camera
const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight; // the canvas default
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement); //滑鼠做控制
camera.position.z = 2;
controls.update();

//light
const color = 0xFFFFFF;
const intensity = 2;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);
```

```
function resizeRendererToDisplaySize(renderer) {  
    const needResize =  
        (canvas.width !== canvas.clientWidth) ||  
        (canvas.height !== canvas.clientHeight);  
  
    if (needResize) {  
        renderer.setSize(canvas.clientWidth,  
            canvas.clientHeight, false);  
    }  
    return needResize;  
}  
  
//動畫  
function render(time) {  
    time *= 0.001; // convert time to seconds (??)  
  
    if (resizeRendererToDisplaySize(renderer)) {  
        camera.aspect = canvas.clientWidth / canvas.clientHeight; //更新 render  
        camera.updateProjectionMatrix();  
    }  
  
    /*  
    cube.rotation.x = time;  
    cube.rotation.y = time * 3;  
    */  
  
    //mesh.rotation.x = time;  
    cube.rotation.y = time;  
    //group.rotation.y = time ;  
  
    camera.aspect = canvas.clientWidth / canvas.clientHeight; //更新畫布  
    camera.updateProjectionMatrix();  
  
    renderer.render(scene, camera);  
  
    requestAnimationFrame(render);  
}  
requestAnimationFrame(render);  
//renderer.render(scene, camera);
```

```
</script>  
</body>  
</html>  
</html>
```

呈現轉動動畫



★心得

本周上課延續上周所學的基本模型來延伸，加入響應式設計，透過視窗大小的改變，可影響模型的變換，也能設計按比例縮放，並加入不同的體積練習，每個體積對應之程式參數皆不同，呈現的樣式可彈性的自由變換，除了模型的體積變換外，也能讓模型呈現透視的狀態，有不一樣的風格與趣味。燈光的部分，則解釋每個燈的作用與特性，讓作品可以呈現更好。

因此作業 2 就由上述筆記練習之程式構成機器人。而我認為較難的部分則是燈與物體之間的旋轉，需要透過 group 來做連接的關係，讓子物體能圍繞主物體來做旋轉的動畫，需要花較多時間理解物體之間的關係與程式內容。

● Week 12

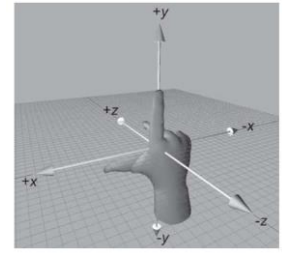
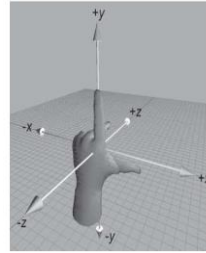
1. 3D Transformation(L7)講解

A. left-handed coordinate space(左手定則)(左圖)

→ 使用的軟體，EX: CSS、Canvas、Unity

B. right-handed coordinate space(右手定則)(右圖)

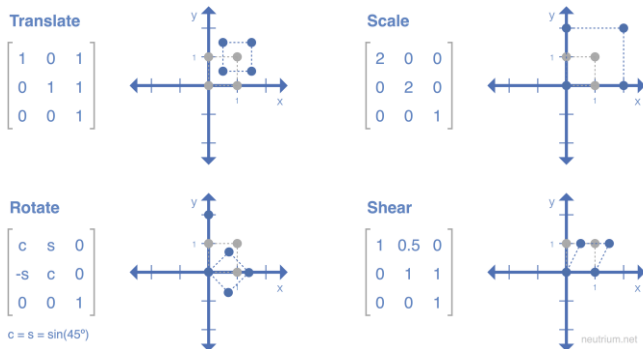
→ 使用的軟體，EX: Three.js、Unreal Engine



C. Affine Transformations

→ 指在幾何中，對一個向量空間進行一次線性轉換並接上一個平移，轉換為另一個向量空間。

如圖所示：



D. Homogeneous Coordinates(齊次座標)

→ 齊次座標是所有電腦圖形系統的關鍵。

所有標準的變換（旋轉、平移、縮放）都可以使用 4x4 矩陣來實現。

對於正交視圖，可以為向量保持 $w=0$ ，對於點保持 $w=1$ 。

對於透視，我們需要透視除法。

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

E. Translation Matrix

→ We can also express translation using a 4 x 4 matrix T in homogeneous coordinates.

$$T(d_x, d_y, d_z)(P) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + d_x \\ y + d_y \\ z + d_z \\ 1 \end{bmatrix}$$

F. Rotation about the x, y, z axis

Z:

$$R_{z\theta}(P) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \\ z \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} x \cos \theta - y \sin \theta \\ x \sin \theta + y \cos \theta \\ z \end{bmatrix}$$

X:

$$R = R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Y:

$$R = R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

G. Scaling、Shear

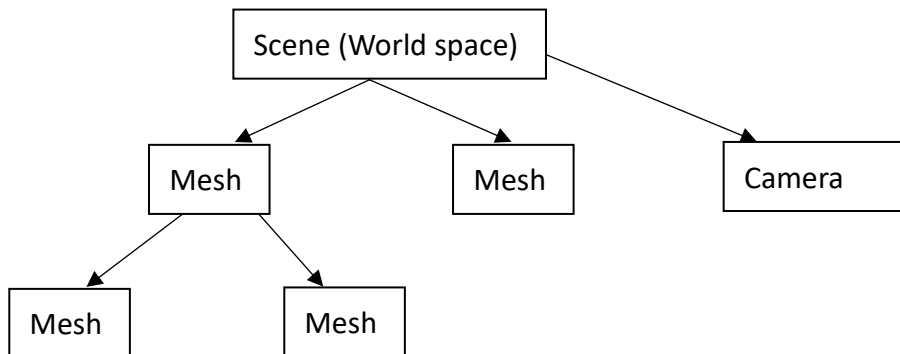
scaling

$$S = S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

shear

$$H(\theta) = \begin{bmatrix} 1 & \cot \theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

H. 3D 模型整體架構



2. 課堂練習_程式(LAB)

→ Create an animation: A point light at center of sun, Earth rotate around sun, Moon rotate around earth.

```

<!DOCTYPE html>
<html>

<head>
  <title>My first three.js app</title>
  <style>
    html,
    body {
      margin: 0;
      height: 100%;
    }

    #c {
      width: 100%;
      height: 100%;
      display: block;
    }
  </style>
</head>
  
```

```
<body>
  <canvas id="c" width="1000" height="1000"></canvas>
  <script type="module">
    import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
    import { OrbitControls } from
"https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

    const canvas = document.getElementById('c');
    const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });
    const scene = new THREE.Scene();
    /* set up default scene */

    let sun = new THREE.Mesh(
      new THREE.SphereBufferGeometry(2), new THREE.MeshPhongMaterial({ color:
'orange' })
    );
    sun.position.set(0, 1, 0);
    //scene.add(sun);

    let earth = new THREE.Mesh(
      new THREE.SphereBufferGeometry(1), new THREE.MeshLambertMaterial({ color:
'steelblue' })
    );
    earth.position.set(5, 0, 0);
    //sun.add(earth);

    let moon = new THREE.Mesh(
      new THREE.SphereBufferGeometry(0.3), new THREE.MeshLambertMaterial({ color:
'white' })
    );
    moon.position.set(-4, 0, 0);

    //依序加入場景之中

    scene.add(sun);
    sun.add(earth);
    earth.add(moon);

    var floor = new THREE.Mesh(
      new THREE.PlaneBufferGeometry(20, 10, 20, 20),
```

```
new THREE.MeshLambertMaterial({ color: 'green', wireframe: true })
);

//camera
const fov = 75;
const aspect = canvas.clientWidth / canvas.clientHeight; // the canvas default
const near = 0.1;
const far = 100;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
const controls = new OrbitControls(camera, renderer.domElement); //滑鼠做控制
camera.position.z = 2;
controls.update();

//light
const color = 0xFFFFFF;
const intensity = 2;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);

earth.updateMatrix();
earth.updateWorldMatrix();

console.log(earth.getWorldPosition(new THREE.Vector3()));
console.log(earth.localToWorld(moon.position));

function resizeRendererToDisplaySize(renderer) {
    const needResize =
        (canvas.width !== canvas.clientWidth) ||
        (canvas.height !== canvas.clientHeight);

    if (needResize) {
        renderer.setSize(canvas.clientWidth,
            canvas.clientHeight, false);
    }
    return needResize;
}
```

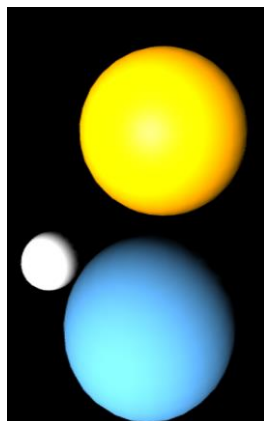
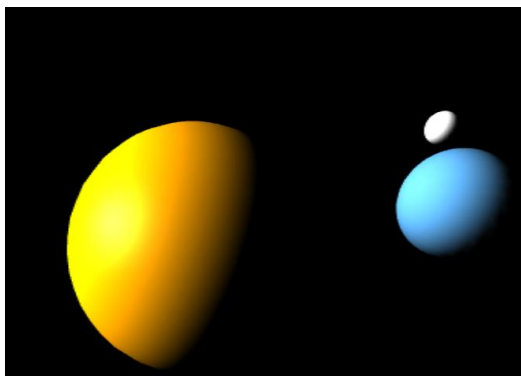
//動畫

```
function render(time) {  
    time *= 0.001; // convert time to seconds  
  
    sun.rotation.y = time*0.1; // 太陽的轉動(地球會跟著轉)  
  
    earth.rotation.y = time*0.1; // 地球的轉動(月球會跟著轉)  
  
    //moon.position.set(2 * Math.cos(time), 0, 2 * Math.sin(time));  
  
    controls.update(); // Update controls here  
    camera.aspect = canvas.clientWidth / canvas.clientHeight;  
  
    camera.updateProjectionMatrix();  
  
    renderer.render(scene, camera);  
  
    requestAnimationFrame(render);  
}  
requestAnimationFrame(render);
```

</script>

</body>

</html>



★ 心得

今日課程較少在程式操作練習方面，而是較多 3D 的理論解釋，例如:3D 模型的架構、矩陣、左(右)手定則等等，其中花較多時間在解釋左(右)手定則，對於 3D 模型的幫助與理解，而每個軟體所使用的都不盡相同，但大致上的概念是一樣的。

後半段則是操作課堂練習的動畫(地球繞太陽，月球繞地球)，這個 3D 模型的關鍵在於安排各個天體的運動軌跡和速度，以及適當地設置光源能產生逼真的光影效果，特別是需要使用到上次所學的 group 能使子物體繞著主物體旋轉的概念(與燈光概念相似)。

總結來說，製需要熟悉 3D 建模和動畫製作的基本原理和技術，透過這幾堂課所使用之練習，展示太陽、地球和月球之間微妙的運動關係。

●Week 13

1. HW2 作業發表，分享每個人的作業成果。(程式碼較多)

My Homework 2

```
<!DOCTYPE html>

<html>

<head>

  <title>My first three.js app</title>

  <style>

    /*網頁在不同大小的顯示器上(從桌面電腦到平板電腦到手機)都能正常顯示。
    將 HTML 和 body 的高度設置為 100% 使它們填滿整個視窗。
    將其顯示模式設置為 block。*/

    html,
    body {
      margin: 0;
      height: 100%;
    }

    #c {
      width: 100%;
      height: 100%;
      display: block;
    }

  </style>
</head>

<body>

  <canvas id="c" width="1000" height="1000"></canvas>

  <script type="module">///允許使用 import 載入使用 three.js 提供的功能和物件

    import * as THREE from "https://cdn.skypack.dev/three@0.132.2";
    import { OrbitControls } from "https://cdn.skypack.dev/three@0.132.2/examples/jsm/controls/OrbitControls.js";

    /* 3D 繪圖需建立以下三元素

    const renderer;

    const scene;

    const camera;

    */
```

```
const canvas = document.getElementById('c');

const renderer = new THREE.WebGLRenderer({ antialias: true, canvas });

const scene = new THREE.Scene(); //創建場景

////////////////////////////////////

//相機設定

const fov = 75;          //相機視野角度 75 度

const aspect = canvas.clientWidth / canvas.clientHeight;

const near = 0.1;        //相機 near plane

const far = 30;          //相機 far plane

const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);

const controls = new OrbitControls(camera, renderer.domElement);


var group = new THREE.Group();

group.add(camera);


camera.position.z = 6;

////////////////////////////////////

//燈光基本設定

/*

const color = 0xFFFFFF; //光的顏色

const intensity = 0.3; //光的強度

const light = new THREE.DirectionalLight(color, intensity); //光的模式

light.position.set(-1, 2, 4); //光的初始位置


const color1 = 0xFFFFFF;

const intensity1 = 0.5;

const light1 = new THREE.DirectionalLight(color, intensity);

light1.position.set(1, -2, 4);


const color2 = 0xFFFFFF;

const intensity2 = 0.5;

const light2 = new THREE.DirectionalLight(color, intensity);

light2.position.set(1, -2, -5);


scene.add(light);

scene.add(light1);

group.add(camera);

*/

////////////////////////////////////

//加入光的旋轉動畫
```

```
const color3 = 0xffffff; //白光
const PointLight1 = new THREE.PointLight(color3, 1);
var light_group1 = new THREE.Group();
light_group1.add(PointLight1);
PointLight1.position.set(0, 6, 0);
scene.add(light_group1);

const color4 = 0xfffacd; //floralwhite
const PointLight2 = new THREE.PointLight(color4, 1);
var light_group2 = new THREE.Group();
light_group2.add(PointLight2);
PointLight2.position.set(6, 5, 0);
scene.add(light_group2);

////////////////////////////////////
//機器人模型建立(以下為各部位詳細參數)
//身體
const geometry_01 = new THREE.BoxGeometry(4, 4, 2);
const material_01 = new THREE.MeshPhongMaterial({ color: 0x7fffd4 });
const mesh_01 = new THREE.Mesh(geometry_01, material_01);
mesh_01.position.x = 0;
mesh_01.position.y = 0;
scene.add(mesh_01);

//帽子
const geometry_00 = new THREE.ConeGeometry(1.1, 1, 80);
const material_00 = new THREE.MeshPhongMaterial({ color: 0xffff00 });
const mesh_00 = new THREE.Mesh(geometry_00, material_00);
mesh_00.position.x = 0;
mesh_00.position.y = 4;
mesh_00.position.z = 0;
scene.add(mesh_00);

//頭
const geometry_02 = new THREE.SphereGeometry(1.2, 30, 30);
const material_02 = new THREE.MeshPhongMaterial({ color: 0xAA2598 });
const mesh_02 = new THREE.Mesh(geometry_02, material_02);
mesh_02.position.x = 0;
mesh_02.position.y = 2.8;
```

```
mesh_02.position.z = 0;
scene.add(mesh_02);

//脖子
const geometry_023 = new THREE.ConeGeometry(1, 2, 80);
const material_023 = new THREE.MeshPhongMaterial({ color: 0xffc0cb });
const mesh_023 = new THREE.Mesh(geometry_023, material_023);
mesh_023.position.x = 0;
mesh_023.position.y = 3;
mesh_023.position.z = 0;
scene.add(mesh_023);

//右耳
const geometry_021 = new THREE.SphereGeometry(0.5, 30, 30);
const material_021 = new THREE.MeshNormalMaterial({ color: 0xffff00 });
const mesh_021 = new THREE.Mesh(geometry_021, material_021);
mesh_021.position.x = 1;
mesh_021.position.y = 2.8;
mesh_021.position.z = 0;
scene.add(mesh_021);

//右眼
const geometry_021_1 = new THREE.SphereGeometry(0.2, 30, 30);
const material_021_1 = new THREE.MeshNormalMaterial({ color: 0xffff00 });
const mesh_021_1 = new THREE.Mesh(geometry_021_1, material_021_1);
mesh_021_1.position.x = 0.4;
mesh_021_1.position.y = 3;
mesh_021_1.position.z = 1;
scene.add(mesh_021_1);

//左眼
const geometry_021_2 = new THREE.SphereGeometry(0.2, 30, 30);
const material_021_2 = new THREE.MeshNormalMaterial({ color: 0xffff00 });
const mesh_021_2 = new THREE.Mesh(geometry_021_2, material_021_2);
mesh_021_2.position.x = -0.4;
mesh_021_2.position.y = 3;
mesh_021_2.position.z = 1;
scene.add(mesh_021_2);

//左耳
```

```
const geometry_022 = new THREE.SphereGeometry(0.5, 30, 30);
const material_022 = new THREE.MeshNormalMaterial({ color: 0xffff00 });
const mesh_022 = new THREE.Mesh(geometry_022, material_022);
mesh_022.position.x = -1;
mesh_022.position.y = 2.8;
mesh_022.position.z = 0;
scene.add(mesh_022);

//嘴巴
const geometry_024 = new THREE.TorusKnotGeometry(0.2, 0.03, 150, 10);
const material_024 = new THREE.MeshNormalMaterial({ color: 0xffff00 });
const mesh_024 = new THREE.Mesh(geometry_024, material_024);
mesh_024.position.x = 0;
mesh_024.position.y = 2.5;
mesh_024.position.z = 1.1;
scene.add(mesh_024);

//手關節右(肩膀)
const geometry_03 = new THREE.SphereGeometry(0.8, 30, 30);
const material_03 = new THREE.MeshPhongMaterial({ color: 0xff1493 });
const mesh_03 = new THREE.Mesh(geometry_03, material_03);
mesh_03.position.x = 2.5;
mesh_03.position.y = 1;
mesh_03.position.z = 0;
scene.add(mesh_03);

//手關節左(肩膀)
const mesh_04 = new THREE.Mesh(geometry_03, material_03);
mesh_04.position.x = -2.5;
mesh_04.position.y = 1;
mesh_04.position.z = 0;
scene.add(mesh_04);

//圓環_1
const geometry_09 = new THREE.TorusGeometry(5, 0.2, 16, 100);
const geometry_d = new THREE.WireframeGeometry(geometry_09);
const material_09 = new THREE.MeshBasicMaterial({ color: 0x0000cd, wireframe: true });
//const material_09 = new THREE.MeshPhongMaterial({ color: 0x22FFAA });
const mesh_09 = new THREE.Mesh(geometry_09, material_09);
mesh_09.rotation.x = 90;
```

```
mesh_09.position.x = 0;
mesh_09.position.y = -0.5;
mesh_09.position.z = 0.5;
scene.add(mesh_09);

//圓環_1_1
const geometry_093 = new THREE.TorusGeometry(5, 0.2, 16, 100);
const geometry_k = new THREE.WireframeGeometry(geometry_093);
const material_093 = new THREE.MeshBasicMaterial({ color: 0x0000cd, wireframe: true });
//const material_09 = new THREE.MeshPhongMaterial({ color: 0x22FFAA });
const mesh_093 = new THREE.Mesh(geometry_093, material_093);
mesh_093.rotation.x = 80;
mesh_093.position.x = 0;
mesh_093.position.y = -0.5;
mesh_093.position.z = 0.5;
scene.add(mesh_093);

//圓環_1_2
const geometry_094 = new THREE.TorusGeometry(5, 0.2, 16, 100);
const geometry_l = new THREE.WireframeGeometry(geometry_094);
const material_094 = new THREE.MeshBasicMaterial({ color: 0x0000cd, wireframe: true });
//const material_09 = new THREE.MeshPhongMaterial({ color: 0x22FFAA });
const mesh_094 = new THREE.Mesh(geometry_094, material_094);
mesh_094.rotation.x = 60;
mesh_094.position.x = 0;
mesh_094.position.y = -0.5;
mesh_094.position.z = 0.5;
scene.add(mesh_094);

//圓環_2
const geometry_010 = new THREE.TorusGeometry(5, 0.2, 16, 100);
const geometry_e = new THREE.WireframeGeometry(geometry_010);
const material_010 = new THREE.MeshBasicMaterial({ color: 0xadd8e6, wireframe: true });
//const material_09 = new THREE.MeshPhongMaterial({ color: 0x22FFAA });
const mesh_010 = new THREE.Mesh(geometry_010, material_010);
mesh_010.rotation.y = 70;
mesh_010.position.x = 0;
mesh_010.position.y = -0.5;
mesh_010.position.z = 0.5;
scene.add(mesh_010);
```

//圓環_2_1

```
const geometry_013 = new THREE.TorusGeometry(5, 0.2, 16, 100);
const geometry_g = new THREE.WireframeGeometry(geometry_013);
const material_013 = new THREE.MeshBasicMaterial({ color: 0xadd8e6, wireframe: true });
//const material_09 = new THREE.MeshPhongMaterial({ color: 0x22FFAA });
const mesh_013 = new THREE.Mesh(geometry_013, material_013);
mesh_013.rotation.y = 30;
mesh_013.position.x = 0;
mesh_013.position.y = -0.5;
mesh_013.position.z = 0.5;
scene.add(mesh_013);
```

//圓環_2_2

```
const geometry_014 = new THREE.TorusGeometry(5, 0.2, 16, 100);
const geometry_h = new THREE.WireframeGeometry(geometry_014);
const material_014 = new THREE.MeshBasicMaterial({ color: 0xadd8e6, wireframe: true });
//const material_09 = new THREE.MeshPhongMaterial({ color: 0x22FFAA });
const mesh_014 = new THREE.Mesh(geometry_014, material_014);
mesh_014.rotation.y = -70;
mesh_014.position.x = 0;
mesh_014.position.y = -0.5;
mesh_014.position.z = 0.5;
scene.add(mesh_014);
```

//身體 2

```
const geometry_05 = new THREE.BoxGeometry(2, 1, 2);
const material_05 = new THREE.MeshPhongMaterial({ color: 0x7fffd4 });
const mesh_05 = new THREE.Mesh(geometry_05, material_05);
mesh_05.position.x = 0;
mesh_05.position.y = -2;
scene.add(mesh_05);
```

//身體 3(中間層次)

```
const geometry_16 = new THREE.BoxGeometry(3, 2, 1);
const material_16 = new THREE.MeshPhongMaterial({ color: 0xb0c4de });
const mesh_16 = new THREE.Mesh(geometry_16, material_16);
mesh_16.position.x = 0;
mesh_16.position.y = 0.5;
mesh_16.position.z = 0.6;
```

```
scene.add(mesh_16);
```

```
//身體 4(裝飾 扣環)
```

```
const geometry_16_1 = new THREE.TorusKnotGeometry(0.3, 0.03, 150, 10);
```

```
const material_16_1 = new THREE.MeshNormalMaterial({ color: 0xb0c4de });
```

```
const mesh_16_1 = new THREE.Mesh(geometry_16_1, material_16_1);
```

```
mesh_16_1.position.x = 0;
```

```
mesh_16_1.position.y = 0.5;
```

```
mesh_16_1.position.z = 1.1;
```

```
scene.add(mesh_16_1);
```

```
//身體 5(裝飾 扣環)
```

```
const geometry_16_2 = new THREE.TorusKnotGeometry(0.2, 0.03, 150, 10);
```

```
const material_16_2 = new THREE.MeshNormalMaterial({ color: 0xb0c4de });
```

```
const mesh_16_2 = new THREE.Mesh(geometry_16_2, material_16_2);
```

```
mesh_16_2.position.x = 0;
```

```
mesh_16_2.position.y = 0.5;
```

```
mesh_16_2.position.z = 1.1;
```

```
scene.add(mesh_16_2);
```

```
//身體 7(裝飾 扣環)
```

```
const geometry_16_3 = new THREE.TorusKnotGeometry(0.2, 0.03, 150, 10);
```

```
const material_16_3 = new THREE.MeshNormalMaterial({ color: 0xb0c4de });
```

```
const mesh_16_3 = new THREE.Mesh(geometry_16_3, material_16_3);
```

```
mesh_16_3.position.x = 0.9;
```

```
mesh_16_3.position.y = 0.5;
```

```
mesh_16_3.position.z = 1.1;
```

```
scene.add(mesh_16_3);
```

```
//身體 8(裝飾 扣環)
```

```
const geometry_16_4 = new THREE.TorusKnotGeometry(0.2, 0.03, 150, 10);
```

```
const material_16_4 = new THREE.MeshNormalMaterial({ color: 0xb0c4de });
```

```
const mesh_16_4 = new THREE.Mesh(geometry_16_4, material_16_4);
```

```
mesh_16_4.position.x = -0.9;
```

```
mesh_16_4.position.y = 0.5;
```

```
mesh_16_4.position.z = 1.1;
```

```
scene.add(mesh_16_4);
```

```
//腰帶
```

```
const geometry_099 = new THREE.BoxGeometry(4.3, 0.5, 2.3);
```

```
const material_099 = new THREE.MeshPhongMaterial({ color: 0xff00ff });
const mesh_099 = new THREE.Mesh(geometry_099, material_099);
mesh_099.position.x = 0;
mesh_099.position.y = -1.3;
scene.add(mesh_099);

//扣環
const geometry_015 = new THREE.IcosahedronGeometry(0.65);
const material_015 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });
const mesh_015 = new THREE.Mesh(geometry_015, material_015);
mesh_015.position.x = 2.5;
mesh_015.position.y = -1.3;
scene.add(mesh_015);

//右手_1
const geometry_08 = new THREE.CylinderGeometry(0.45, 0.45, 2.5, 60);
const geometry_c = new THREE.WireframeGeometry(geometry_08);
const material_08 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });
//const material_06 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });
const mesh_08 = new THREE.Mesh(geometry_08, material_08);
mesh_08.position.x = 2.5;
mesh_08.position.y = 0.3;
scene.add(mesh_08);

//左手_1
const geometry_019 = new THREE.CylinderGeometry(0.45, 0.45, 2.5, 60);
const geometry_i = new THREE.WireframeGeometry(geometry_019);
const material_019 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });
//const material_06 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });
const mesh_019 = new THREE.Mesh(geometry_019, material_019);
mesh_019.position.x = -2.5;
mesh_019.position.y = 0.3;
scene.add(mesh_019);

//右手關節
const geometry_081 = new THREE.IcosahedronGeometry(0.65);
const material_081 = new THREE.MeshLambertMaterial({ color: 0x00bfff });
const mesh_081 = new THREE.Mesh(geometry_081, material_081);
mesh_081.position.x = 2.5;
mesh_081.position.y = -1.3;
```

```
scene.add(mesh_081);
```

```
//左手關節
```

```
const geometry_020 = new THREE.IcosahedronGeometry(0.65);  
const material_020 = new THREE.MeshLambertMaterial({ color: 0x00bfff });  
const mesh_020 = new THREE.Mesh(geometry_020, material_020);  
mesh_020.position.x = -2.5;  
mesh_020.position.y = -1.3;  
scene.add(mesh_020);
```

```
//右手_2
```

```
const geometry_082 = new THREE.CylinderGeometry(0.25, 0.25, 2.5, 60);  
const geometry_f = new THREE.WireframeGeometry(geometry_082);  
const material_082 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });  
//const material_06 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });  
const mesh_082 = new THREE.Mesh(geometry_082, material_082);  
mesh_082.position.x = 2.5;  
mesh_082.position.y = -1.6;  
mesh_082.position.z = 1;  
mesh_082.rotation.x = 93;  
scene.add(mesh_082);
```

```
//左手_3
```

```
const geometry_026 = new THREE.CylinderGeometry(0.25, 0.25, 2.5, 60);  
const geometry_j = new THREE.WireframeGeometry(geometry_026);  
const material_026 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });  
//const material_06 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });  
const mesh_026 = new THREE.Mesh(geometry_026, material_026);  
mesh_026.position.x = -2.5;  
mesh_026.position.y = -0.65;  
mesh_026.position.z = 1;  
mesh_026.rotation.x = 205;  
scene.add(mesh_026);
```

```
//右手_手指 3
```

```
const geometry_083 = new THREE.SphereGeometry(0.4, 30, 30);  
const material_083 = new THREE.MeshLambertMaterial({ color: 0xffc0cb });  
const mesh_083 = new THREE.Mesh(geometry_083, material_083);  
mesh_083.position.x = 2.5;  
mesh_083.position.y = -2;
```

```
mesh_083.position.z = 2.3;
mesh_083.rotation.x = 93;
scene.add(mesh_083);

//左手_手指 3
const geometry_0261 = new THREE.SphereGeometry(0.4, 30, 30);
const material_0261 = new THREE.MeshLambertMaterial({ color: 0xffc0cb });
const mesh_0261 = new THREE.Mesh(geometry_0261, material_0261);
mesh_0261.position.x = -2.5;
mesh_0261.position.y = 0.3;
mesh_0261.position.z = 2;
mesh_0261.rotation.x = 93;
scene.add(mesh_0261);

//大聲公_1
const geometry_030 = new THREE.CylinderGeometry(0.09, 0.3, 1, 80);
const material_030 = new THREE.MeshLambertMaterial({ color: 0xffffff });
const mesh_030 = new THREE.Mesh(geometry_030, material_030);
mesh_030.position.x = 2.5;
mesh_030.position.y = -2.3;
mesh_030.position.z = 3.2;
//mesh_030.rotation.x = 93;
scene.add(mesh_030);

//大聲公_1_5
const geometry_032 = new THREE.CylinderGeometry(0.09, 0.2, 1, 80);
const material_032 = new THREE.MeshLambertMaterial({ color: 0xff4500 });
const mesh_032 = new THREE.Mesh(geometry_032, material_032);
mesh_032.position.x = 2.5;
mesh_032.position.y = -2.35;
mesh_032.position.z = 3.2;
//mesh_030.rotation.x = 93;
scene.add(mesh_032);

//大聲公_2(手把)
const geometry_031 = new THREE.CylinderGeometry(0.15, 0.15, 0.67, 80);
const material_031 = new THREE.MeshPhongMaterial({ color: 0x000000 });
const mesh_031 = new THREE.Mesh(geometry_031, material_031);
mesh_031.position.x = 2.5;
mesh_031.position.y = -2.2;
```

```
mesh_031.position.z = 3;
mesh_031.rotation.x = 90;
scene.add(mesh_031);

//指揮棒
const geometry_091 = new THREE.CylinderGeometry(0.15, 0.15, 0.5, 30);
const material_091 = new THREE.MeshLambertMaterial({ color: 0x000000 });
const mesh_091 = new THREE.Mesh(geometry_091, material_091);
mesh_091.position.x = -2.5;
mesh_091.position.y = 0.35;
mesh_091.position.z = 2.5;
//mesh_091.rotation.x = 93;
scene.add(mesh_091);

//指揮棒_1
const geometry_092 = new THREE.CylinderGeometry(0.13, 0.13, 2, 30);
const material_092 = new THREE.MeshNormalMaterial({ color: 0xff0000 });
const mesh_092 = new THREE.Mesh(geometry_092, material_092);
mesh_092.position.x = -2.5;
mesh_092.position.y = 1.2;
mesh_092.position.z = 2.5;
//mesh_091.rotation.x = 93;
scene.add(mesh_092);

//右腳(直)
const geometry_06 = new THREE.CylinderGeometry(0.45, 0.45, 4, 60);
const geometry_a = new THREE.WireframeGeometry(geometry_06);
const material_06 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });
//const material_06 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });
const mesh_06 = new THREE.Mesh(geometry_06, material_06);
mesh_06.position.x = 1.4;
mesh_06.position.y = -3;
scene.add(mesh_06);

//右腳關節
const geometry_061 = new THREE.IcosahedronGeometry(0.65);
//const geometry_k = new THREE.WireframeGeometry(geometry_061);
//const material_061 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });
const material_061 = new THREE.MeshLambertMaterial({ color: 0x00bfff });
const mesh_061 = new THREE.Mesh(geometry_061, material_061);
```

```

mesh_061.position.x = 1.4;
mesh_061.position.y = -3.5;
scene.add(mesh_061);

//左腳(直)
const geometry_07 = new THREE.CylinderGeometry(0.45, 0.45, 4, 60);
const geometry_b = new THREE.WireframeGeometry(geometry_07);
const material_07 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });
//const material_07 = new THREE.MeshLambertMaterial({ color: 0x00FF00 });
const mesh_07 = new THREE.Mesh(geometry_07, material_07);
mesh_07.position.x = -1.4;
mesh_07.position.y = -3;
scene.add(mesh_07);

//左腳關節
const geometry_071 = new THREE.IcosahedronGeometry(0.65);
//const geometry_k = new THREE.WireframeGeometry(geometry_061);
//const material_061 = new THREE.MeshBasicMaterial({ color: 0xffe4b5, wireframe: true });
const material_071 = new THREE.MeshLambertMaterial({ color: 0x00bfff });
const mesh_071 = new THREE.Mesh(geometry_071, material_071);
mesh_071.position.x = -1.4;
mesh_071.position.y = -3.5;
scene.add(mesh_071);

//右腳底
const geometry_011 = new THREE.BoxGeometry(1, 0.3, 1.3);
const material_011 = new THREE.MeshPhongMaterial({ color: 0x6a5acd });
const mesh_011 = new THREE.Mesh(geometry_011, material_011);
mesh_011.position.x = 1.4;
mesh_011.position.y = -5;
scene.add(mesh_011);

//左腳底
const geometry_012 = new THREE.BoxGeometry(1, 0.3, 1.3);
const material_012 = new THREE.MeshPhongMaterial({ color: 0x6a5acd });
const mesh_012 = new THREE.Mesh(geometry_012, material_012);
mesh_012.position.x = -1.4;
mesh_012.position.y = -5;
scene.add(mesh_012);

////////////////////////////////////

```

```
//重設畫布解析度(當螢幕比例調整時, 會隨之改變)

function resizeRendererToDisplaySize(renderer) {

    const needResize =

        (canvas.width !== canvas.clientWidth) ||

        (canvas.height !== canvas.clientHeight);

    if (needResize) {

        renderer.setSize(canvas.clientWidth,

            canvas.clientHeight, false);

    }

    return needResize;

}

//場景基本 render 設置

function render(time) {

    time *= 0.001; // convert time to seconds

    //mesh.rotation.x = time;

    //mesh.rotation.y = time;

    controls.update(); // Update controls here

    camera.aspect = canvas.clientWidth / canvas.clientHeight;

    camera.updateProjectionMatrix();

    PointLight1.rotation.x = time;

    PointLight2.rotation.y = time*0.9;

    light_group1.rotation.x = time;

    light_group2.rotation.y = time*0.9;

    renderer.render(scene, camera);

    requestAnimationFrame(render);

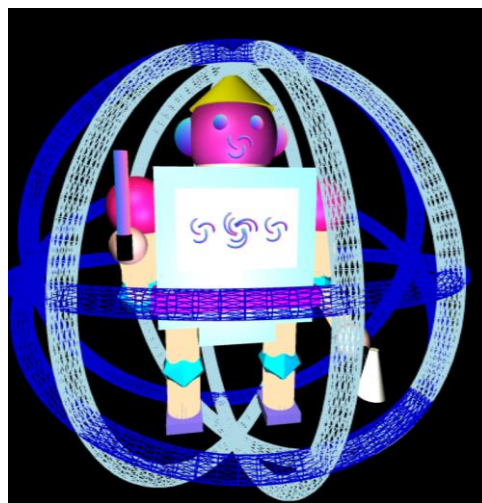
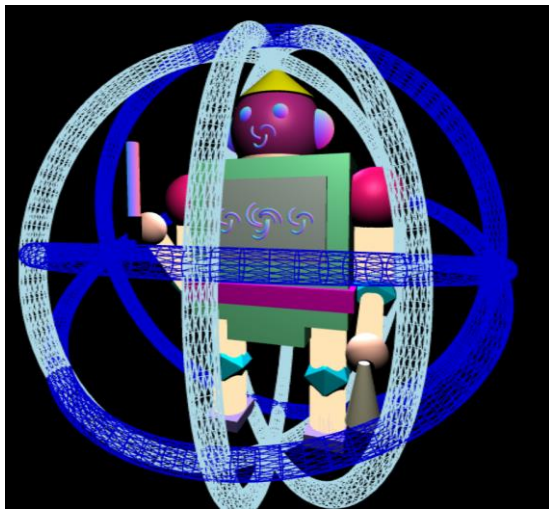
}

requestAnimationFrame(render);
```

</script>

</body>

</html>



2. L8 lighting and shading(介紹燈的種類、模型的材質)

2-1. Light sources

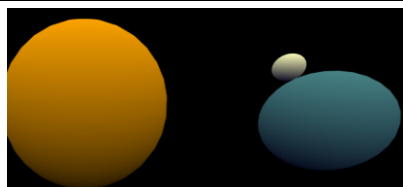
a. Ambient Light

```
const intensity = 1;
const light = new THREE.AmbientLight(0xfffacd);
scene.add(light);
```



b. Hemisphere Light

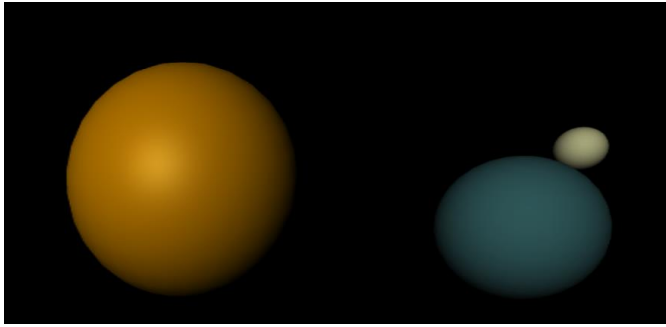
```
const intensity = 5;
const light = new THREE.HemisphereLight(0xffffbb, 0x080820, 1);
scene.add(light);
```



c. Point Light

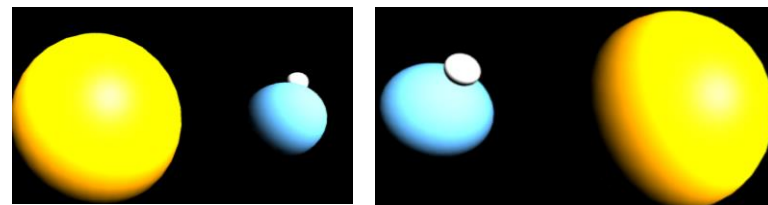
```
const intensity = 100;
const light = new THREE.PointLight(0xffffbb, 1, 100);
light.position.set(20,20,20);
```

```
scene.add(light);
```



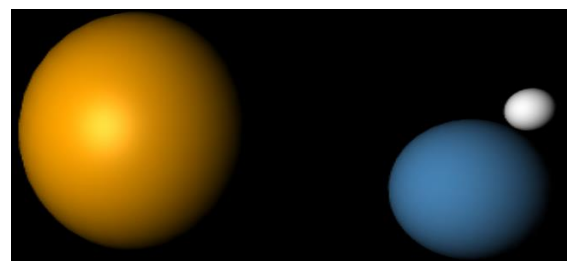
d. Directional Light

```
const light = new THREE.DirectionalLight( 0xffffff, 3 );  
light.position.set(5,5,5);  
scene.add(light);
```



e. Spot Light

```
const intensity = 100;  
const light = new THREE.SpotLight(0xffffff);  
light.position.set(5,5,5);  
scene.add(light);
```

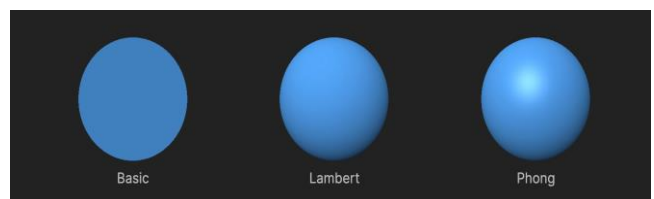


2-2. Light and Material Interaction

a. shading → 光線和材質之間的互動，可以形成不同的顏色或色調，讓模型有不同的變化。

b. 常見會需要思考的幾個點

1. 燈光的種類
2. 材質的樣式
3. 觀看的角度位置
4. 表面的材質(燈光照射是否反光或折射)

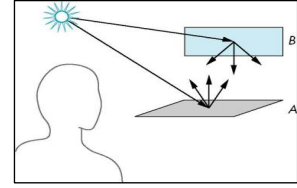


c. 光線照射物體時，可能有一些光會部分被吸收，一些部分被散射（反射）；被反射的量決定了物體的顏色和亮度。Ex:在白光下，表面看起來是紅色，是因為紅色光的成分被反射，其餘被吸收。

→因此，反射的光以取決於表面的光滑度和方向性來做反射。

d. Scattering 散射

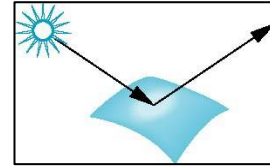
光照射物體 A 與 B，有一些光會被物體吸收，有一些會被反射。



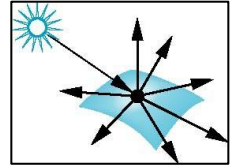
e. Surface Types 表面

→表面會影響光的照射的樣子。

→表面越光滑，光越集中在表面上，再反射；表面粗糙，光會向所有方向反射。



smooth surface



rough surface

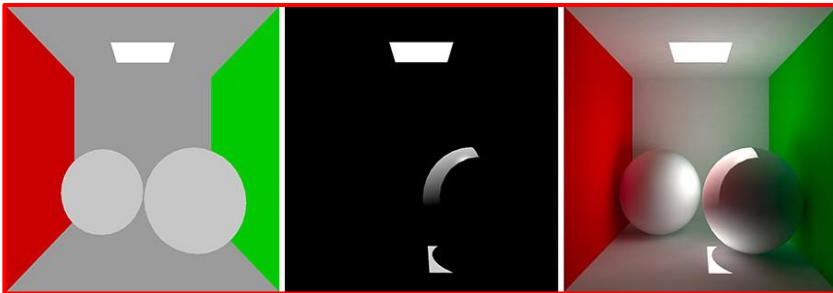
f. Local vs Global Rendering

Local Rendering → 區域性的渲染圖，因此是一個比較快且不差(顏色看似正確)的渲染方案。

Global Rendering → 需要涉及所有物體和光源的全部計算(更接近正確顏色)。

g. Global illumination(全域光照)(需要花較多時間，但效果較好)

→沒有照明(左)，只有直接光源(中)，和有間接光源的全域光照(右)的表現



★心得

這週的上半堂課程主要在欣賞每個同學的 HW2 成果，我看到有些同學很有創意，在製作模型方面，絞盡腦汁，每個機器人都各具特色，甚至看到有同學利用貼圖的方式包裝機器人，有些連背景都加入，彷彿在那個場景當中；有些同學更用心，花更多時間在做機器人的動畫，讓機器人更加生動。

而後半段課程較少有實際操作的練習，多為課堂口述 3D 概念，內容多為前幾堂提到的概念，加以延伸，特別在燈光與材質介紹，材質與燈光之間的影響，也會影響模型外觀與整體的觀看感受，綜合來說，燈光與材質在 3D 中佔有很重要的部分，下堂課會繼續講更深入的概念。

●Week 14

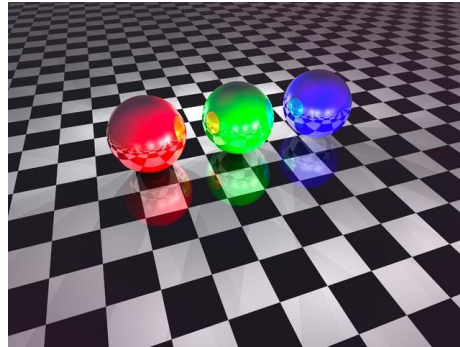
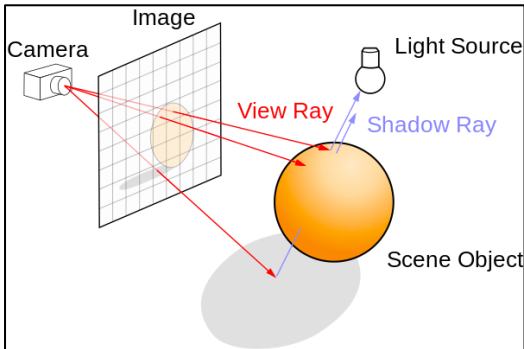
1. LightMap

當烘焙一張光照天突時，場景內的靜態物件會基於光的影響算出一張貼圖成果，並疊在場景物件之上建立照明關係。

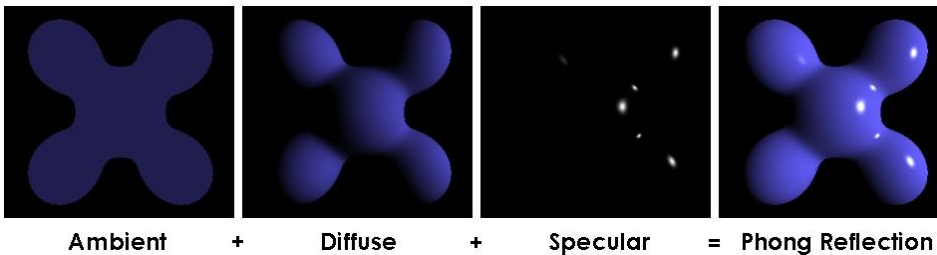
2. 預計算全域光照

雖然傳統的靜態光照貼圖無法在遊戲執行時改變場景光照條件，但預先計算的即時全域光照系統能幫忙及時運算複雜的場景光源互動。

3. 光線追蹤(Ray-Tracing)



4. Phong Model



Ambient + Diffuse + Specular = Phong Reflection

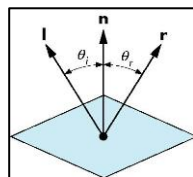
A simple model that can be computed rapidly

Has three components : Diffuse 、Specular 、Ambient

5. Ideal Reflector

Normal is determined by local orientation.

Angle of incidence = angle of reflection.



6. Ambient Light(Background Light)

Ambient light is the result of multiple interactions between (large) light sources and the objects in the environment.

Amount and color depend on both the color of the light(s) and the material properties of the object.

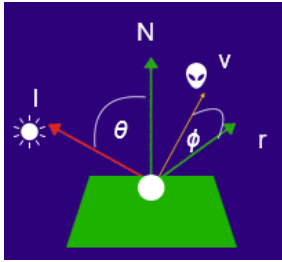
$$Ambient(I_a) = k_a \times I_a$$

object light

7. Lambertian Surface(Diffuse Surface)

Perfectly diffuse reflector Light scattered equally in all directions.

Amount of light reflected is proportional to the vertical component of incoming light.



8. Specular Surfaces

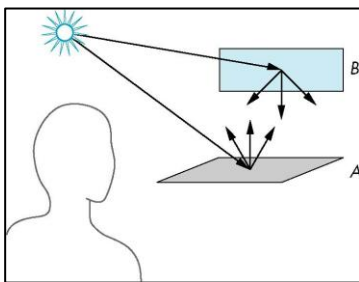
Most surfaces are neither ideal diffusers nor perfectly specular (ideal reflectors).

Smooth surfaces show specular highlights due to incoming light being reflected in directions concentrated close to the direction of a perfect reflection.

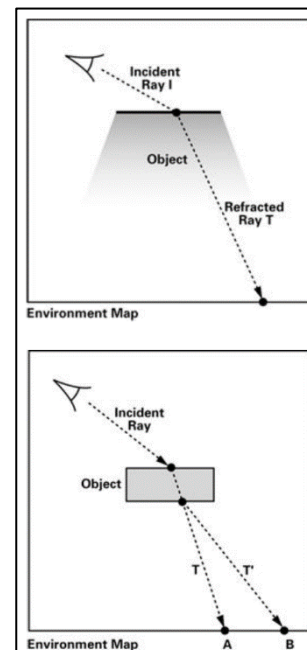
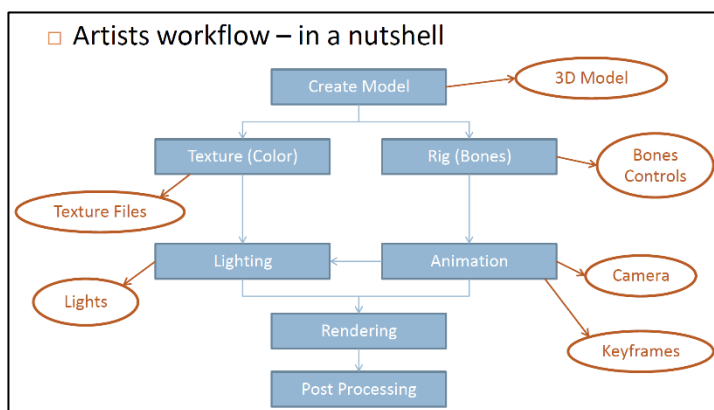
9. Distance Terms

The light from a point source that reaches a surface is inversely proportional to the square of the distance between them

The constant and linear terms soften the effect of the point source.



10. What is 3D Graphics?

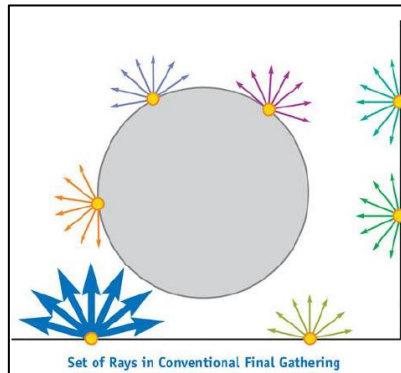
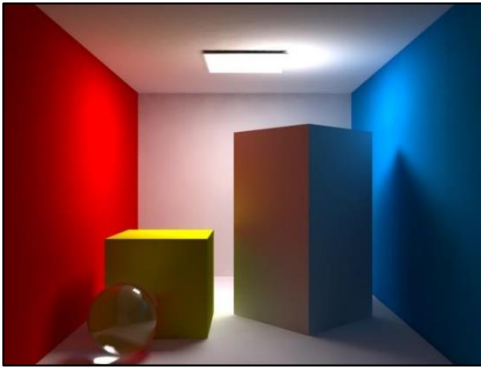


11. Refractions (Real time rendering)

Refraction is complex so fake refractions are hard to notice.

12. Indirect Illumination (Real time rendering)

Use a lot of fast rasterized “renders”.



★心得

本次上課較少操作程式練習的部分，反而有更多 3D 動畫的觀念講述，特別是在渲染(rendering)與燈光這部分，也提到以前傳統在做 3D 時，電腦的效能及如何製作，到現階段的建模過程，簡單地提及卻印象深刻，因為以前的年代無法像現代一樣，科技發達普遍，晶片、GPU 也已相對便宜，能更有更好的效能製作出動畫呈現，而這些技術演變到現在，更有光線追蹤，讓 3D 更為生動且真實。

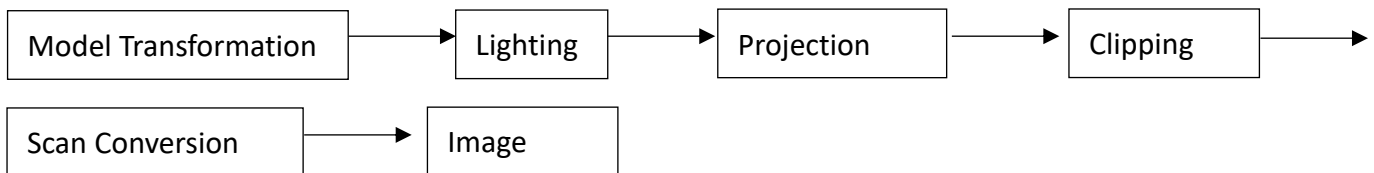
●Week 15

1. Real-time Rendering (Parallel Processing)

Metric	SGI IR	Nvidia GF4
Triangles/sec	13 million	136 million
Pixels/sec		4.8 million
Texture memory	64MB	128MB
Bump mapping	No	Yes
Programmable Vertex engine?	No	Yes
Programmable Pixel engine?	No	Yes
Physical size	Mini-Fridge	Video Cassette
Cost	\$100,000	\$300

Major advance

2. Real-time Rendering (Graphics Pipeline)



a. Model Transformation

Transform each triangle from object space to world space

b. Lighting

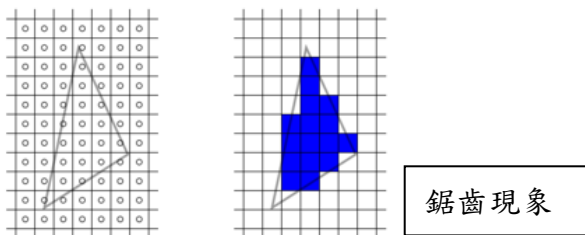
Computation is based on angles between light source, object and camera (details later)

c. Projection

Transform triangles from world space to camera space (orthogonal transformation)

d. Scan Conversion

Drawing the triangles in 2D./ Scanning horizontal scan line for each triangle.



● Viewing and Projection

→在二維表面上呈現三維世界的藝術家必須使用錯覺系統來創造對空間，深度和運動的印象。



● 3D illusion example



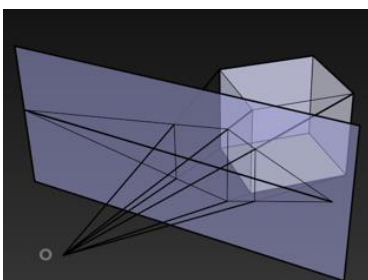
●Vanishing Points

→ 物體上不與投影面平行的平行線在投影中匯聚於一點（消失點）



●Perspective Projection(透視投影)

→Rays of light travel from the object, through the picture plane, and to the viewer's eye.

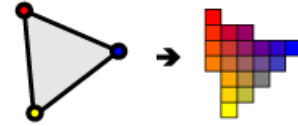
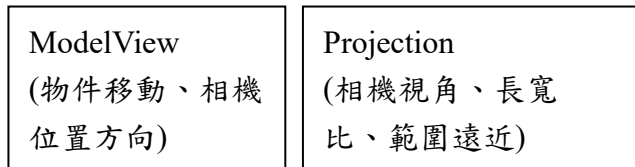


●Computer Viewing

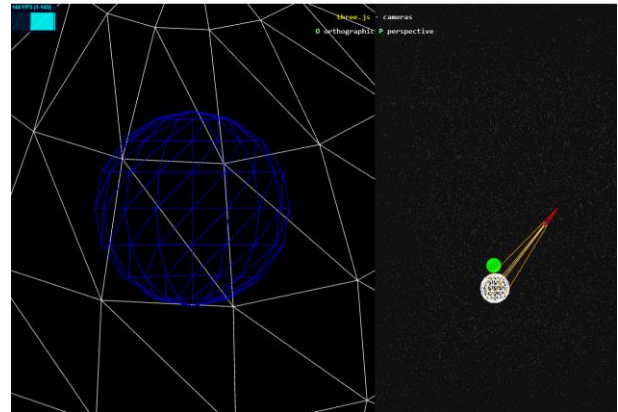
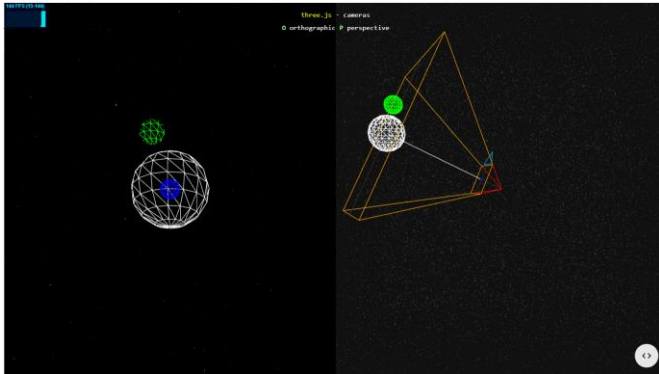
→分為三個方面

1. 相機位置與鏡頭方向(Positioning the camera)
2. 相機可視空間，包含視角、長寬比、遠近範圍(Select a lens)
3. 可視空間外的裁剪(Clipping)

Vertex array→Vertex Shader→Triangle Assembly→Rasterization→Fragment Shader→Framebuffer



●Field of view (https://threejs.org/examples/#webgl_camera)



★心得

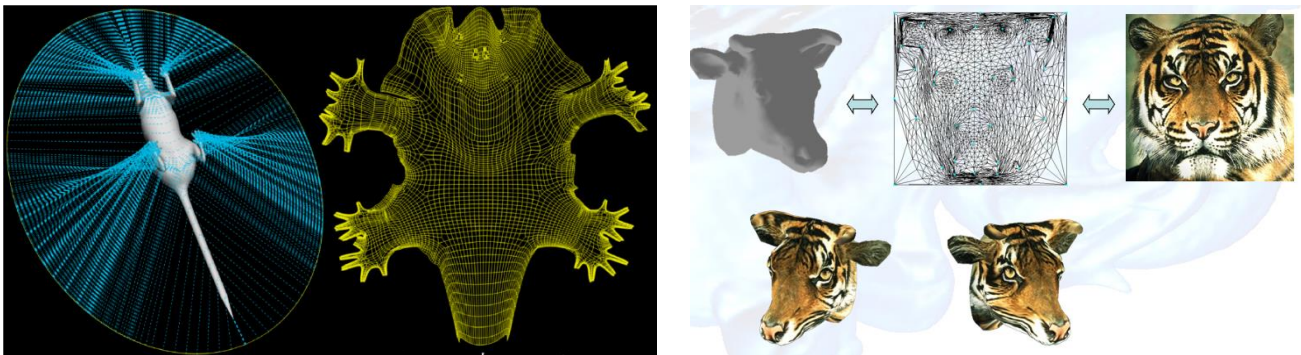
今天課程也較少程式實際練習的部分，多數為講解概念。其中介紹到相機視角，決定觀看者如何觀察 3D 場景的重要因素。透過調整相機的位置、角度和焦距，可以創造出不同的視覺效果，例如俯視、仰視、近景和遠景。除了視角外，透視投影是一種重要的技術，它模擬了人眼看物體的方式。透視投影使得遠處的物體看起來較小，近處的物體看起來較大，因此創造出深度感。在 3D 繪圖中是很重要的觀念，因為它能夠使場景看起來更真實和立體。綜合運用這些技術，可以創建出栩栩如生的 3D 圖像和動畫，提升視覺體驗。

●Week 16

1. texture mapping(範例)



2. Mesh Parameterization → 利用網格方式，把材質貼圖貼上



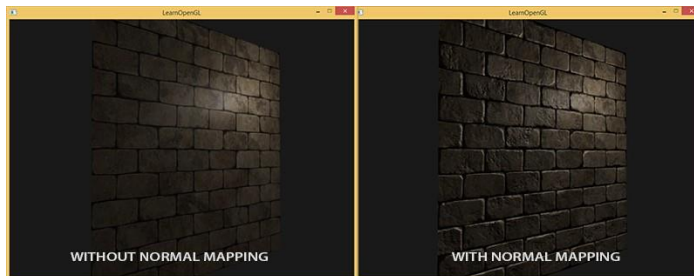
3. Normal mapping

(1)Bump mapping

→利用紋理貼圖來干擾每個像素的法向量，可以讓 3D model 呈現更多細節的錯覺。

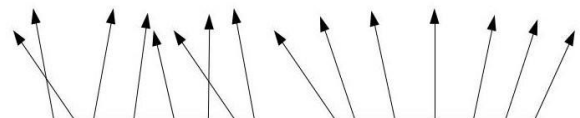
→法線貼圖是把每個像素儲存方向的圖像。這些方向稱為法線。

EX：

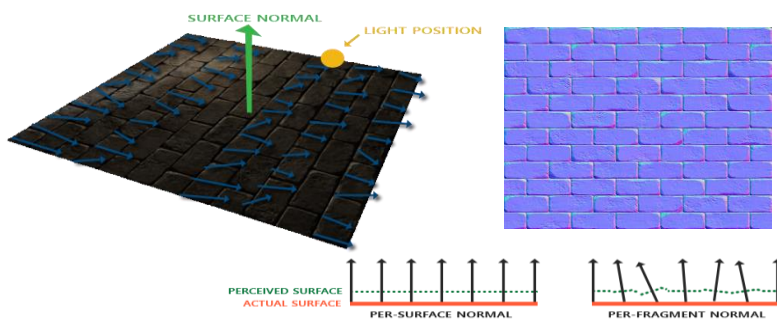


Bump Mapping

Regular Lighting



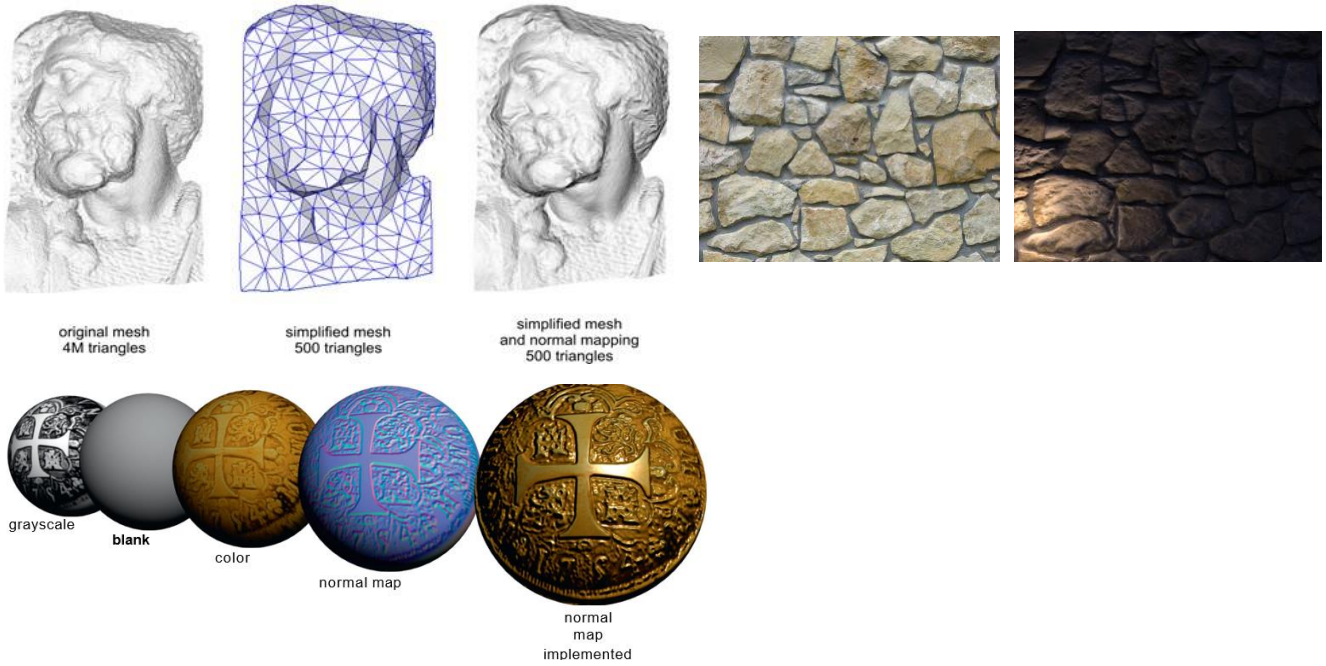
4.per pixel lighting



5.Normal mapping + low poly

→法線貼圖通常用於在低解析度模型上模擬高解析度細節，可以渲染相同數量的多邊形（即低解析度網格），但在計算光照時使用高解析度網格的法線，可以使模型有更大的深度、真實感和平滑度。

EX :



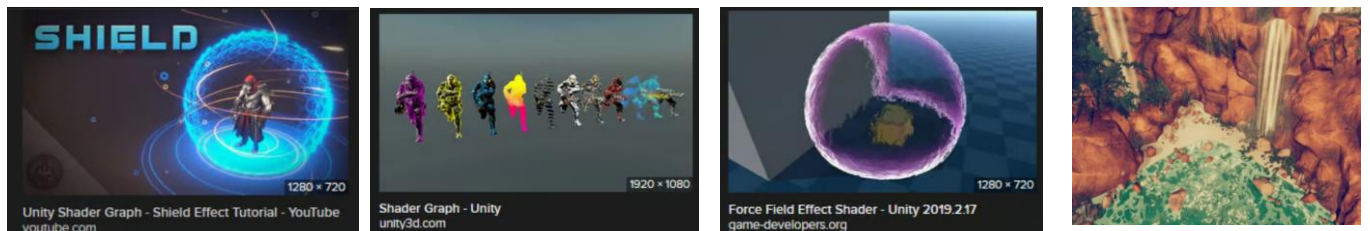
6.What is Shader

- 著色器是用 GLSL 編寫的小程序，運行在 GPU 上。
- 對於每種材質，可以指定兩種不同類型的著色器。

7.Shader Pipelined Process

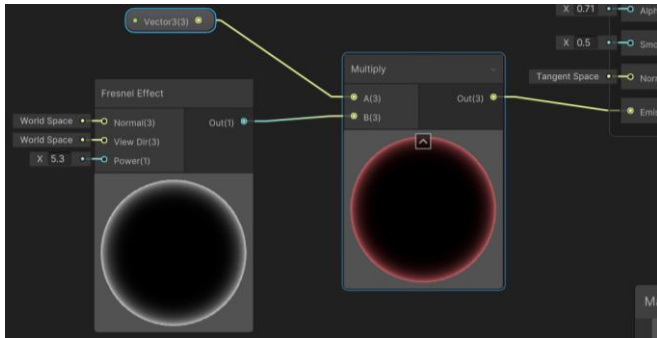


8.應用→shader 可以產生不同的效果

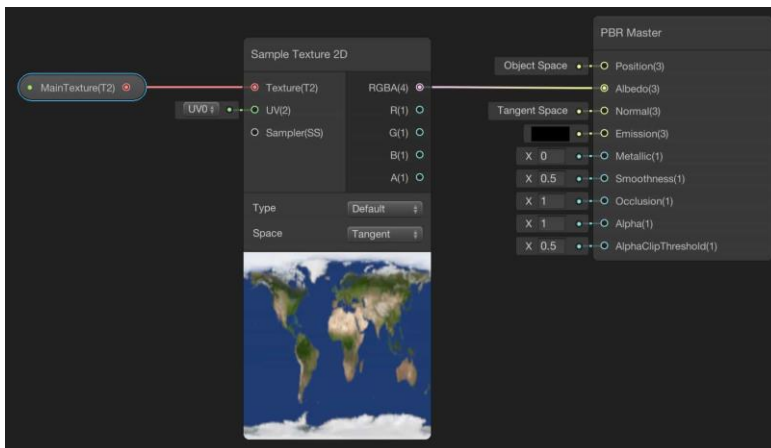


9.Unity 操作練習

rim effect



Texture mapping



★心得

今天是這學期電腦圖學的最後一堂課，除了前期利用 html 程式製作模型和各種特效外，後半段學習到許多 3D 的基本理論，例如：材質、光影、紋理、視覺效果等等。而課程中學習到的這些知識，可以幫助我更好理解圖形渲染的細節和優化技術。電腦圖學的應用領域非常廣泛，包括遊戲開發、電影製作、虛擬現實、增強現實等；是一個不斷發展的領域，新技術和新方法層出不窮。因此，這門課程讓我對未來職業發展有了更多的選擇和思考，並深刻意識到持續學習的重要性，保持對新技術的敏感度，才能在這個快速發展的領域中保持競爭力。

#6/10(一)端午節放假、6/17(一)期末考週停課